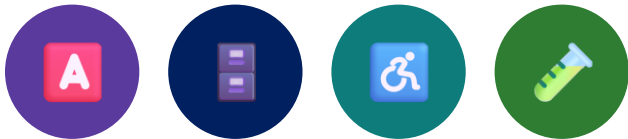


*"A component tree that renders is not proof
-- only a PostgreSQL row surviving a restart
is."*

MODULE 50

Capstone Frontend and Persistence

Complete the Northstar CRM agent journey: an accessible Angular search, profile, timeline, and interaction form, backed by durable PostgreSQL persistence via Spring Data JPA, consuming Module 49's API contracts exactly.





MODULE 50 OVERVIEW

Complete a usable Angular CRM journey backed by durable PostgreSQL persistence via Spring Data JPA, consuming Module 49's API exactly.

Agents need an accessible search, profile, timeline, and interaction form with durable PostgreSQL storage. A UI that looks done without database proof or accessibility basics fails the capstone gate.

DURATION & LAB



4 Hours Theory

Angular review, UI development, backend integration, PostgreSQL persistence, Spring Data JPA, and testing.



2 Hours Lab

lab50-crm: Angular journey, migrations, JPA mappings, typed HttpClient service, a11y, restart-durability proof.



Scenario

Search Amina (CUS-1001), open her timeline, record an interaction with lab-request-001.

MODULE ARC



Journey First

Search, profile, timeline, form -- one coherent Angular flow, not a component zoo.



PostgreSQL Proof

A toast is not persistence -- prove the row survives an API restart.



Accessibility Is Graded

Labels, keyboard order, and aria-live announcements are rubric marks, not polish.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours lab = 6 hours. No pass without UI proof, PostgreSQL row proof, accessibility basics, and one automated UI/component check.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to build and prove a full-stack CRM journey for the Northstar CRM.

1. Build typed Angular services against Module 49's API contracts.
2. Implement accessible forms with labels, focus management, and live-region alerts.
3. Complete JPA and PostgreSQL mappings with deliberate relationship ownership.
4. Create repeatable, versioned PostgreSQL migrations.
5. Handle loading, empty, success, invalid, unauthorized, and outage states.
6. Verify UI-to-database flow with restart durability.
7. Automate component tests and one critical-path Selenium E2E journey.
8. Document a reproducible frontend-and-persistence demo for Module 52.

KEY TAKEAWAY These eight objectives are drawn directly from the graded lab -- you will build and prove every layer of the search-to-interaction journey.



MODULE 50

CAPSTONE BUILD FRONTEND AND PERSISTENCE

Module Learning Objectives

By the end of this module, you will be able to design, build, and integrate the frontend and persistence layers of the Capstone application.



BUILD A RESPONSIVE ANGULAR FRONTEND

Develop reusable components, routes, and services to create a user-friendly and scalable UI.



INTEGRATE WITH REST APIS

Connect the Angular application with Spring Boot REST APIs using HttpClient for full CRUD operations.



IMPLEMENT DATA PERSISTENCE

Design entities and repositories using Spring Data JPA and persist data in PostgreSQL.



ENSURE DATA INTEGRITY AND SECURITY

Apply validation, transactions, and security best practices across the frontend and persistence layers.



OPTIMIZE PERFORMANCE

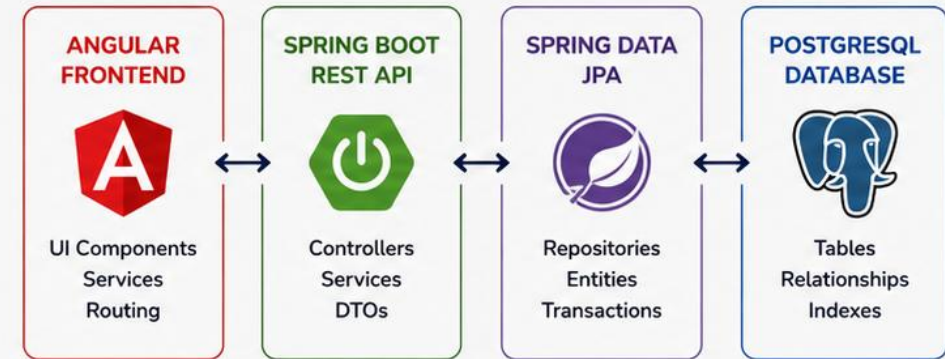
Implement pagination, sorting, and efficient queries to deliver a high-performance application.



VALIDATE END-TO-END WORKFLOWS

Verify create, read, update, and delete workflows through integration and UI testing.

CAPSTONE ARCHITECTURE OVERVIEW



KEY OUTCOMES



Full-stack integration from UI to database



Reliable data persistence with PostgreSQL



Optimized queries and application performance



Secure and validated data workflows



Production-ready, scalable, and maintainable code



High-quality UI/UX with enterprise standards

CAPSTONE PROGRESS REVIEW

Module 48 froze the plan; Module 49 shipped the backend slice; this module attaches the Angular UI and proves persistence.



Module 48: The Plan

Architecture, NFRs, backlog, and ADRs frozen for CAP-12 before any code was written.



Module 49: The Backend

A validated REST API, a transactional service, and a versioned Kafka event, all tested.



Module 50: The Frontend

An accessible Angular journey, wired to that exact API, backed by durable PostgreSQL rows.



Nothing Reinvented

This module consumes Module 49's DTOs and status codes exactly -- no silent contract drift.

KEY TAKEAWAY Any drift from Module 49's contract discovered today is a signal to re-check docs/backend-demo.md, not to improvise a new shape.



MODULE 50

CAPSTONE BUILD FRONTEND AND PERSISTENCE

Capstone Progress Review

We have completed the planning, backend services, and integration foundation. We are now building the frontend and persistence layers to complete our full-stack solution.

CAPSTONE JOURNEY



WHAT WE HAVE BUILT SO FAR



Project Planning & Architecture

Defined requirements, data flows, and system architecture.



Backend Services

Built core Spring Boot microservices and REST APIs.



Security Foundation

Implemented authentication and authorization with JWT.



Kafka Integration

Event-driven communication and asynchronous processing.



External Integrations

Integrated with external systems and validated workflows.



WHAT WE WILL BUILD IN THIS MODULE



ANGULAR FRONTEND

Build responsive UI with components, routing, and services.



PERSISTENCE LAYER

Design entities, repositories using Spring Data JPA and PostgreSQL.



END-TO-END FLOW

Connect UI to APIs and database to complete CRUD operations.



TESTING & VALIDATION

Unit, integration, and UI testing for reliability and quality.



PERFORMANCE & QUALITY

Optimize queries, handle pagination, sorting, and improve performance.



BEST PRACTICES

Apply clean code, security, and enterprise best practices.



OUR GOAL: Deliver a complete, secure, scalable, and high-performance full-stack application ready for deployment.

FRONTEND BUILD OBJECTIVES

What the Angular layer must deliver before Lab 50 can be marked a pass.



Findable

Search resolves Amina and Ravi reliably, with a real not-found state for CUS-9999.



Navigable

Routing carries the selected customer through profile and timeline without a full reload.



Accessible

Labels, keyboard order, and role=alert/aria-live are graded rubric lines, not polish.



Provable

Component tests plus one Selenium critical-path journey, not just a live demo click-through.

KEY TAKEAWAY A frontend that only looks finished in a demo, with no accessibility or automated proof, is explicitly not done.



MODULE 50

CAPSTONE BUILD FRONTEND AND PERSISTENCE

Frontend Build Objectives

Build a responsive, secure, and maintainable Angular application that delivers a seamless user experience and integrates efficiently with the backend REST APIs.



BUILD USER-CENTRIC UI

Develop intuitive, responsive, and accessible user interfaces using reusable Angular components.



INTEGRATE WITH REST APIS

Consume backend APIs using HttpClient to perform CRUD operations and handle data efficiently.



MANAGE AUTHENTICATION & SECURITY

Implement JWT-based authentication, route guards, and HTTP interceptors for secure communication.



IMPLEMENT FORMS & VALIDATION

Build template-driven and reactive forms with client-side validation and server-side error handling.



ENHANCE UX WITH STATE & FEEDBACK

Manage loading, empty, and error states to provide clear feedback and a smooth user experience.



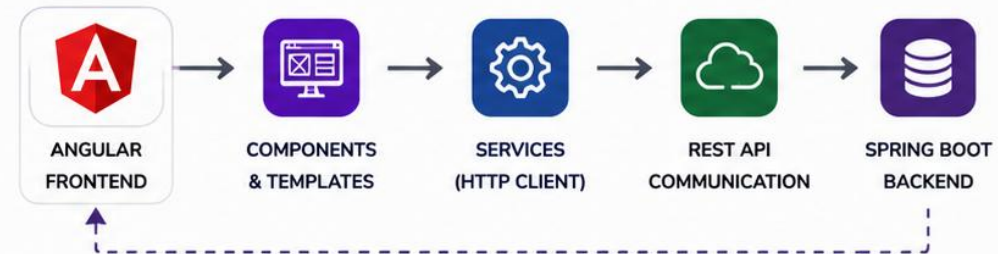
ENSURE PERFORMANCE & BEST PRACTICES

Follow Angular best practices, optimize performance, and maintain clean, modular, and testable code.



GOAL: Deliver a fast, reliable, and user-friendly Angular frontend that provides an excellent experience and integrates seamlessly with the backend services.

FRONTEND ARCHITECTURE OVERVIEW



KEY FRONTEND CAPABILITIES



RESPONSIVE DESIGN

Works seamlessly across desktop, tablet, and mobile devices.



ROUTING

Implement feature-based routing with route guards and lazy loading.



STATE MANAGEMENT

Use Signals / RxJS to manage UI state and reactive data flows.



DATA VALIDATION

Client-side validation with meaningful error messages.



ERROR HANDLING

Global error handling and user-friendly error feedback.



REUSABLE & SCALABLE

Build reusable components and follow scalable architecture patterns.



PERSISTENCE BUILD OBJECTIVES

What the PostgreSQL and Spring Data JPA layer must prove before Lab 50 can be marked a pass.



Migrated

Every schema change ships as a versioned Flyway migration -- never a hand-edited table.



Mapped Deliberately

Entity ownership, fetch strategy, and cascade rules chosen on paper, not left to defaults.



Restart-Durable

A row created through the UI must still be there after the API process restarts.



Query-Correct

Repository methods are named for business intent and tested against real PostgreSQL.

KEY TAKEAWAY A toast notification is never accepted as persistence evidence -- only a SQL row that survives a restart counts.



MODULE 50

CAPSTONE BUILD FRONTEND AND PERSISTENCE

Persistence Build Objectives

Design and implement a robust, scalable, and secure persistence layer using Spring Data JPA and PostgreSQL to support reliable data storage, retrieval, and transaction management for the Capstone application.



DESIGN OPTIMIZED DATA MODEL

Design normalized schemas with proper relationships, constraints, and indexing for performance and integrity.



IMPLEMENT ENTITIES & REPOSITORIES

Create JPA entities and Spring Data repositories to enable maintainable and efficient data access.



ENSURE DATA INTEGRITY

Enforce validation, constraints, and transaction boundaries to ensure accurate and consistent data.



OPTIMIZE QUERY PERFORMANCE

Use indexing, pagination, and query optimization techniques to deliver fast and scalable database operations.



SUPPORT SCALABILITY & RELIABILITY

Implement secure connections, proper exception handling, and best practices for high availability and resilience.



ENABLE TESTABILITY & MAINTAINABILITY

Write integration tests and follow clean architecture principles for long-term maintainability.

PERSISTENCE ARCHITECTURE OVERVIEW



SPRING BOOT
APPLICATION



SPRING DATA JPA
(REPOSITORIES)



JPA / HIBERNATE
(ORM)



POSTGRESQL
DATABASE



TABLES &
RELATIONSHIPS

KEY PERSISTENCE CAPABILITIES



DATA MODELING

Normalized tables, foreign keys, constraints, and appropriate data types.



CRUD & REPOSITORIES

Leverage Spring Data JPA for efficient CRUD operations and queries.



TRANSACTION MANAGEMENT

Ensure ACID compliance using @Transactional and transaction boundaries.



QUERY OPTIMIZATION

Use indexing, JPQL, pagination, and query tuning for performance.



DATA INTEGRITY & SECURITY

Enforce validation rules, constraints, roles, and least privilege access.



TESTING & MAINTAINABILITY

Integration testing with PostgreSQL and clean, well-structured code.



GOAL: Deliver a reliable, secure, and high-performance persistence layer that supports the Capstone application's business requirements.



POSTGRESQL

ANGULAR APPLICATION ARCHITECTURE

A quick review of the Angular shape this module builds on -- standalone components, services, and a typed HttpClient, wired through dependency injection.

APPLICATION SHAPE

AppComponent bootstraps standalone CustomerSearch, CustomerProfile, InteractionTimeline, InteractionForm.

Dependency injection wires each component to its service via the constructor or inject().

The Angular Router owns which screen is active -- no component swaps views manually.

provideHttpClient(withInterceptors([...])) wires the typed API layer app-wide.

Standalone bootstrap (excerpt)

```
bootstrapApplication(AppComponent, {  
  providers: [provideHttpClient(withInterceptors([authInterceptor, errorInterceptor])),  
             provideRouter(routes)]  
});
```

WHAT EACH LAYER AVOIDS

- Components never call HttpClient directly -- only injected services do.
- Services never import anything from @angular/router or manipulate the DOM.
- No component subscribes to an Observable without unsubscribing or the async pipe.
- No JPA entity field name is ever hardcoded into a component template.

KEY TAKEAWAY A component that injects HttpClient directly, bypassing the service layer, is exactly the layering mistake this review exists to prevent.



MODULE 50

CAPSTONE BUILD FRONTEND AND PERSISTENCE

Angular Application Architecture

A modular, scalable, and maintainable Angular architecture that promotes separation of concerns, reusability, and efficient integration with backend REST APIs.



COMPONENT LAYER

UI components organized by features and shared modules. Handles presentation and user interactions.



ROUTING LAYER

Configures routes, lazy loading, and navigation guards for feature-based navigation and access control.



SERVICE LAYER

Provides business logic, HTTP communication, and shared functionality across the application.



STATE MANAGEMENT

Manages application state using Signals / RxJS and shared services for reactive data flow.



INTERCEPTORS & GUARDS

HTTP interceptors for auth, logging, and error handling. Route guards for authentication and authorization.



CORE & SHARED MODULES

Includes reusable components, directives, pipes, and utility services used across features.



KEY BENEFITS



Modular & Scalable



High Performance & Lazy Loading



Secure by Design

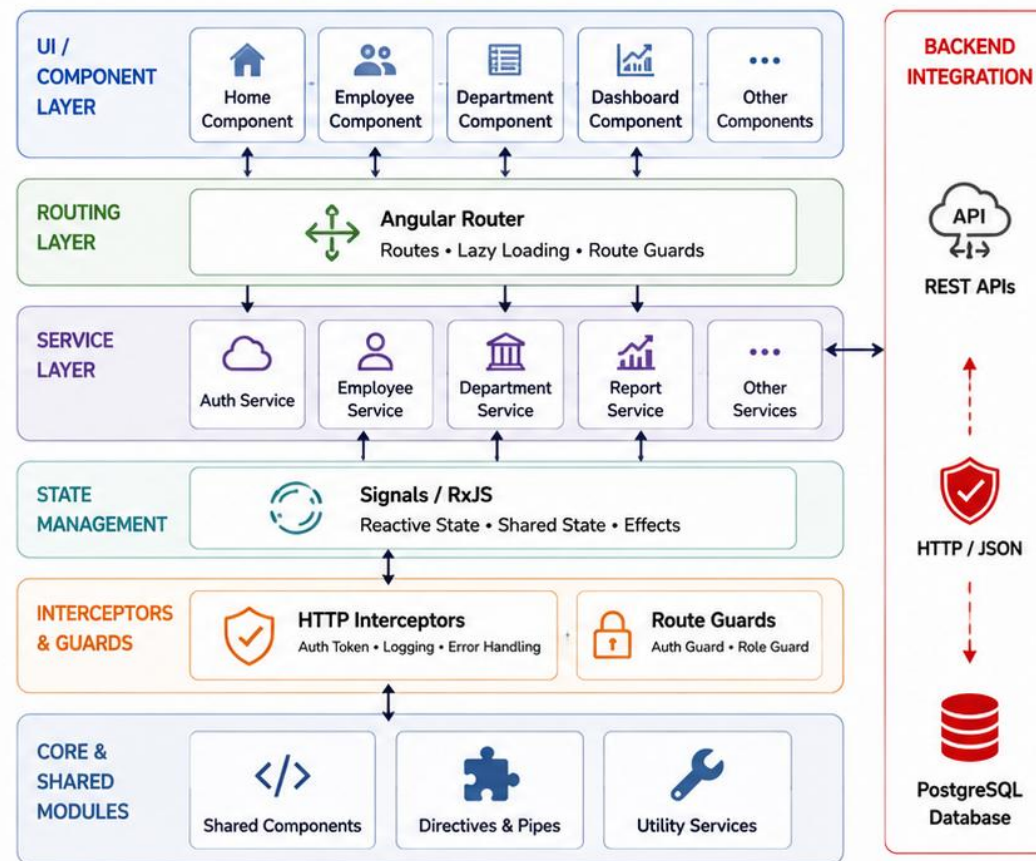


Reusable & Maintainable



Seamless API Integration

ANGULAR APPLICATION ARCHITECTURE



ANGULAR FEATURE STRUCTURE

One feature folder per screen keeps the search-to-interaction journey easy to navigate and to grade.

FOLDER SHAPE

customers/customer-search/ -- component, template, styles, spec.

customers/customer-profile/ -- header details, status badge.

interactions/interaction-timeline/ -- read model, newest first.

interactions/interaction-form/ -- the one write path in the journey.

STRUCTURE DISCIPLINE

- Each feature folder owns its component, not a shared 'components' junk drawer.
- Shared, reusable pieces move to shared/ only once two features need them.
- core/ holds the typed API services, interceptors, and app-wide models.
- No feature folder imports another feature's internal component directly.

Suggested project layout (excerpt)

```
src/app/  
  core/api/customers.service.ts   core/api/interactions.service.ts  
  customers/customer-search/      customers/customer-profile/  
  interactions/interaction-timeline/ interactions/interaction-form/
```

KEY TAKEAWAY A component reached through three folders of unrelated imports is a structure problem this feature layout exists to prevent.



MODULE 50

CAPSTONE BUILD FRONTEND AND PERSISTENCE

Angular Feature Structure

Organize the application by business features.
Each feature is self-contained with components, services, models, and routing.



FEATURE-BASED ORGANIZATION

Group all related files for a feature in a single folder to improve maintainability and scalability.



SELF-CONTAINED FEATURES

Each feature includes components, services, models, and routing specific to that domain.



LAZY LOADING BY FEATURE

Load feature modules on demand to improve performance and initial load time.



REUSABLE SHARED MODULES

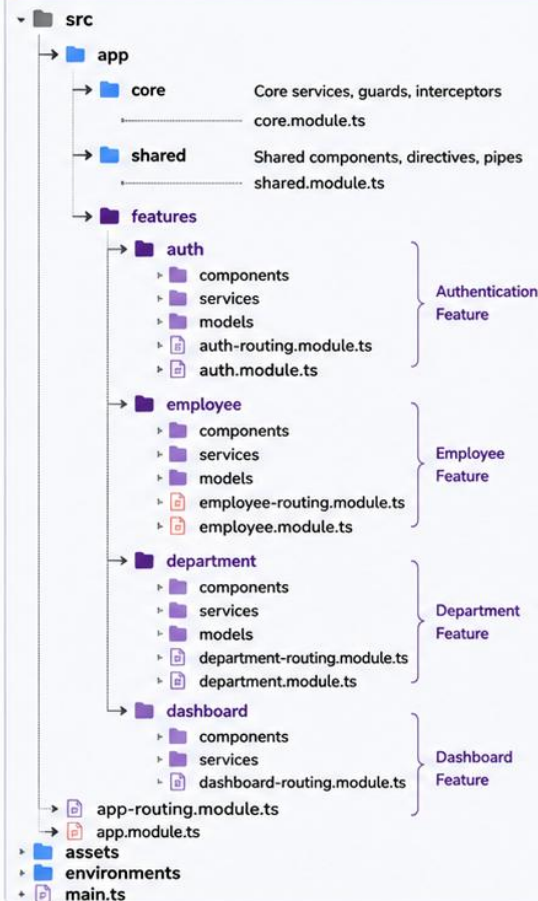
Extract common components, directives, pipes, and utilities into shared modules.



SCALABLE & MAINTAINABLE

Supports growing applications and multiple teams working on different features.

EXAMPLE PROJECT STRUCTURE



FEATURE MODULE CONTENT



Components

UI components for the feature



Services

Business logic and API communication



Models

TypeScript interfaces and models



Routing Module

Feature-specific routes



Feature Module

Feature module with imports and exports

KEY BENEFITS



Improved Scalability

Easily add new features without impacting existing code.



Better Maintainability

Keep related code together and easy to understand.



Performance Optimization

Lazy load features to reduce initial bundle size.



Team Productivity

Teams can work independently on different features.



BEST PRACTICE: Follow feature-based architecture, keep features independent, and share only what is necessary.

REUSABLE COMPONENT STRATEGY

Four components, four jobs -- search, profile, timeline, and form each own one responsibility and hand off cleanly.

COMPONENT RESPONSIBILITIES

CustomerSearch: query input, results list, empty/loading states.

CustomerProfile: header details -- name, id, status badge.

InteractionTimeline: read model, newest first, refreshed after writes.

InteractionForm: the one write path, wired to the typed service.

WHAT EACH COMPONENT AVOIDS

- Search avoids embedding profile-rendering logic.
- Profile avoids fetching interactions itself -- the timeline owns that.
- Timeline avoids duplicating the form's validation rules.
- No component calls HttpClient directly -- only the injected service does.

Composition (excerpt)

```
<app-customer-search (select)="customerId.set($event)" />
@if (customerId()) {
  <app-customer-profile [id]="customerId()" />
  <app-interaction-timeline [customerId]="customerId()" />
  <app-interaction-form [customerId]="customerId()" (saved)="bumpKey()" />
}
```

KEY TAKEAWAY A component that fetches data outside the typed service, or duplicates another component's job, is exactly the layering mistake this review exists to prevent.

Reusable Component Strategy

Reusable components reduce duplication, ensure consistency, and accelerate development across the application.

STRATEGY PILLARS



REUSABILITY

Build components that can be used in multiple features and screens.



CONSISTENCY

Ensure a uniform look and feel across the application.



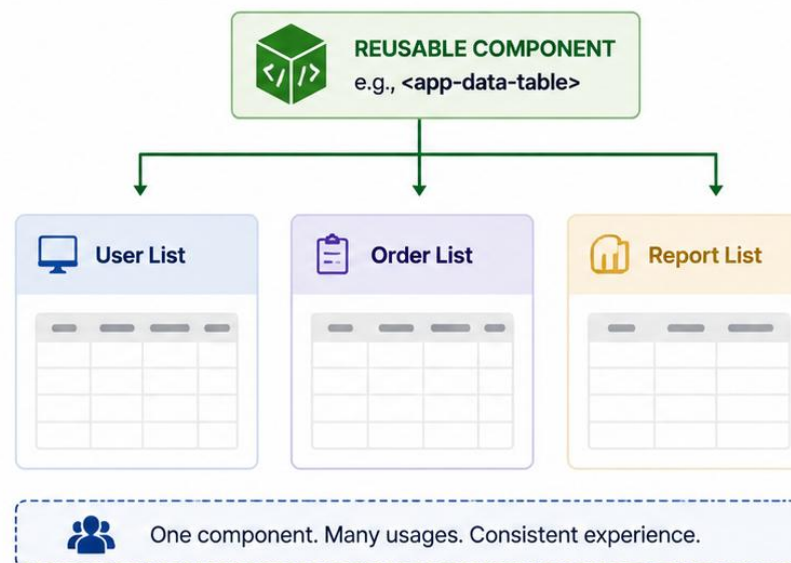
MAINTAINABILITY

Centralize UI logic and styles to simplify updates and bug fixes.



PRODUCTIVITY

Save time by reusing well-tested components.



BEST PRACTICES



SINGLE RESPONSIBILITY

Keep components focused on one job.



CONFIGURABLE VIA INPUTS

Use @Input() to make components flexible and reusable.



COMMUNICATE VIA OUTPUTS

Use @Output() for event communication.



STYLE VIA THEMING

Use shared styles and design tokens for consistency.



DOCUMENT AND TEST

Document component usage and write unit tests.



GOAL: Build a library of high-quality, reusable components that accelerates delivery and maintains a scalable, consistent, and maintainable frontend.



Modular



Consistent



Faster Delivery



Easier Maintenance

ROUTING ARCHITECTURE

The Angular Router carries the selected customer through the journey -- no component swaps screens by hand.

ROUTE SHAPE

/customers -- CustomerSearch, the landing route.

/customers/:id -- CustomerProfile with a resolver loading the customer.

/customers/:id/timeline -- InteractionTimeline nested under the profile.

A route guard blocks navigation to a protected route with no valid session.

ROUTING DISCIPLINE

- The customer id lives in the URL, not only in component state -- routes are shareable.
- A resolver fetches the customer before the component renders, avoiding an empty flash.
- An unmatched id (CUS-9999) resolves to a clear not-found route, not a blank screen.
- Lazy-loaded routes keep the initial bundle focused on the search screen.

Route config (excerpt)

```
export const routes: Routes = [  
  { path: 'customers', component: CustomerSearchComponent },  
  { path: 'customers/:id', component: CustomerProfileComponent,  
    resolve: { customer: customerResolver } },  
];
```

KEY TAKEAWAY A customer profile that only works after clicking search, never from a shared URL, breaks the routing contract this module requires.

TRY IT YOURSELF — EXERCISE 1: MAP FEATURES AND ROUTES

Reinforces: the Angular feature structure and routing architecture you just saw, mapped onto the capstone customer slice before any component is generated.

STEPS (notes/lab50-feature-route-map.md)

Step	What To Do
1 — Feature Folders	List the folders for the customer feature: components, services, models, routes.
2 — Route Table	Write four routes: list, detail, create, and edit — with their path patterns.
3 — Shared vs Feature	Name two components that belong in shared, and say why.
4 — Scope	Paper plan only — do not run ng generate in this pre-lab.

Route table sketch

```
/customers      -> CustomerListComponent  (resolver: page 0)
/customers/:id   -> CustomerDetailComponent (CUS-1001)
/customers/new   /:id/edit -> CustomerFormComponent (reused)
```

TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-feature-route-map.md (workspace: examples/module-50-exercises).

ROUTING ARCHITECTURE

Angular Router enables navigation between views, supports lazy loading, and protects routes.

ROUTING PRINCIPLES



SINGLE PAGE NAVIGATION

Navigate without full page reloads for a seamless user experience.



LAZY LOADING

Load feature modules only when needed to optimize performance.



ROUTE PROTECTION

Guard routes using authentication and role-based authorization.



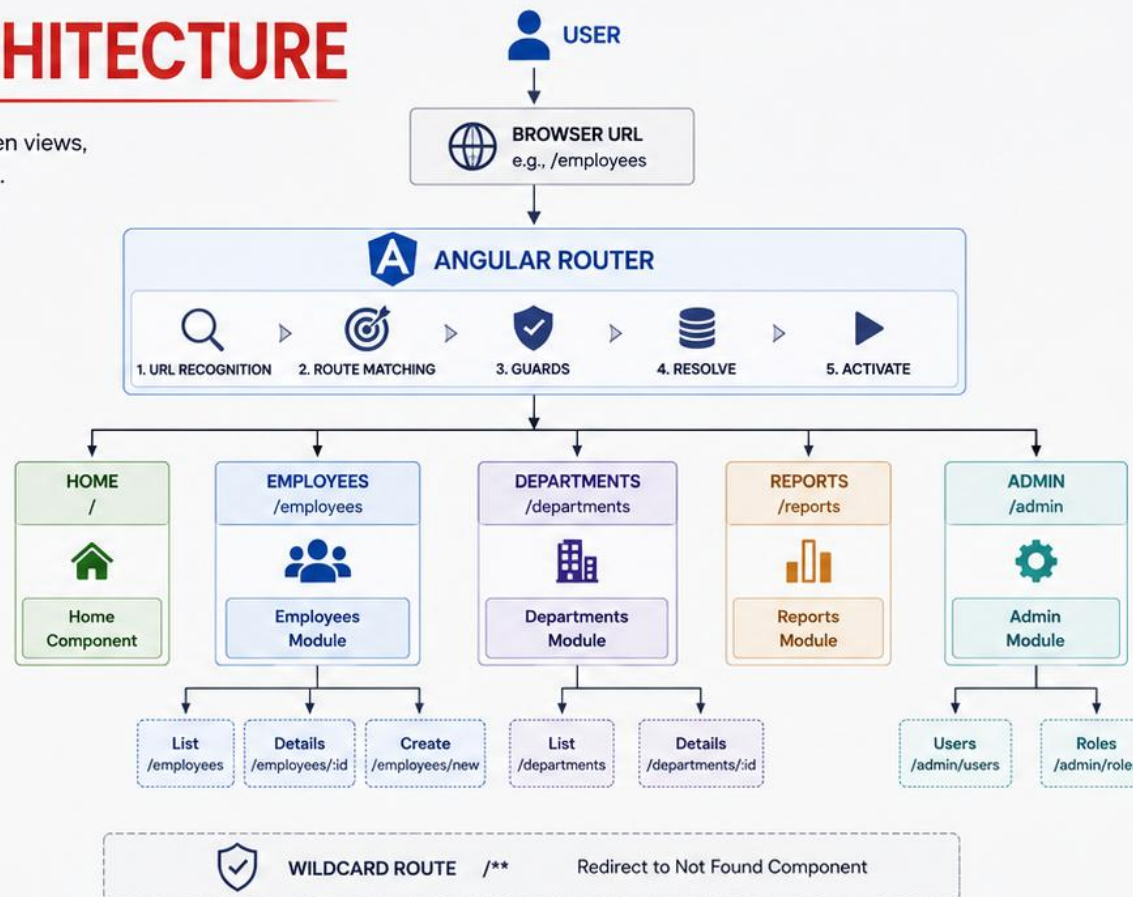
NESTED ROUTES

Organize UI hierarchies with child routes and layouts.



REDIRECTS & WILDCARDS

Handle default navigation, unknown routes, and fallbacks.



BEST PRACTICES



FEATURE-BASED ROUTING

Organize routes by features using lazy-loaded modules.



USE ROUTE GUARDS

Protect sensitive routes with canActivate, canLoad guards.



KEEP ROUTES SIMPLE

Design clear, predictable, and user-friendly URLs.



USE RESOLVERS

Prefetch data before activating routes for better UX.



HANDLE FALLBACKS

Implement wildcard routes and meaningful redirects.



DOCUMENT ROUTES

Maintain a route map for your team and future maintenance.



GOAL: A scalable, secure, and maintainable routing architecture that delivers fast and consistent navigation across the application.



PERFORMANCE

Lazy loading and efficient navigation.



SECURITY

Guarded routes and role-based access.



MAINTAINABILITY

Modular routing that scales with your application.



USER EXPERIENCE

Seamless navigation and fast load times.

STATE MANAGEMENT STRATEGY

Signals hold UI state; RxJS streams server state -- keep them separate so loading, error, and success are never guessed at.

TWO KINDS OF STATE

Server state: the customer, the timeline, and a form-submission result, from RxJS/HttpClient.

UI state: which screen is active, form field values -- held in Angular Signals.

The selected customerId is a signal read by both profile and timeline components.

A submitting signal disables the form's submit button while a request is in flight.

Submitting-state guard (concept)

```
submitting = signal(false);
onSubmit(data: CreateInteractionRequest) {
  if (this.submitting()) return;
  this.submitting.set(true);
  this.interactions.create(this.customerId(), data)
    .pipe(finalize(() => this.submitting.set(false)))
    .subscribe(() => this.saved.emit());
}
```

STATE DISCIPLINE

- Never treat an in-flight request as already succeeded.
- Refetch server state after a write -- do not hand-patch a signal optimistically.
- One source of truth per piece of state -- no duplicated copies drifting apart.
- A superseded, stale Observable emission must be ignored (switchMap, not mergeMap).

KEY TAKEAWAY A double-submit that creates two rows, or a stale response overwriting a fresher one, are both state-management bugs this review exists to prevent.

STATE MANAGEMENT STRATEGY

A well-structured state management approach ensures predictable UI behavior, efficient data flow, and a better developer experience.

STRATEGY PILLARS



SINGLE SOURCE OF TRUTH

Keep application state in a single, predictable location.



PREDICTABLE DATA FLOW

Ensure unidirectional data flow for easier debugging.



REUSABILITY

Share state across components and features efficiently.



PERFORMANCE

Minimize unnecessary updates and re-renders.



MAINTAINABILITY

Structure state for long-term scalability and clarity.

CHOOSE THE RIGHT TOOL

LOCAL / COMPONENT STATE

(Simple & isolated)



Use Angular Signals for local UI state, forms, and component logic.

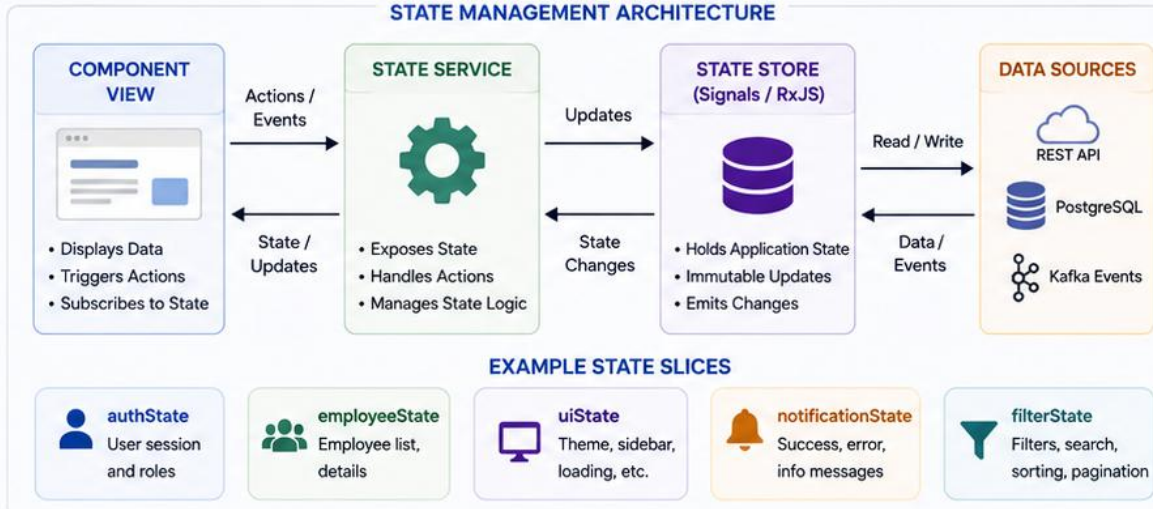
SHARED / GLOBAL STATE

(Complex & shared)



Use RxJS / BehaviorSubject for shared state across components and services.

STATE MANAGEMENT ARCHITECTURE



BEST PRACTICES



KEEP STATE MINIMAL

Store only the data needed for the UI.



USE THE RIGHT TOOL

Signals for local state, RxJS for shared state.



AVOID DIRECT MUTATION

Always update state immutably.



SUBSCRIBE SMARTLY

Unsubscribe properly and use async pipe where possible.



ORGANIZE STATE LOGIC

Keep state logic in services not in components.



DOCUMENT STATE FLOW

Maintain clear documentation for state design.



GOAL: Create a scalable, predictable, and maintainable state management approach for a high-quality enterprise application.



Predictable UI behavior



Better performance and responsiveness



Easier debugging and developer onboarding



Scalable for future features and growth

ANGULAR SERVICE LAYER

Every network call originates from an injectable service -- never from inside a component.

SERVICE RESPONSIBILITIES

CustomersService: search, getCustomer, wraps HttpClient calls to Module 49's API.

InteractionsService: listByCustomer, createInteraction, the one write path.

Both services are @Injectable({ providedIn: 'root' }) -- one instance app-wide.

Services return typed Observable<T>, never raw HttpResponse or any.

SERVICE DISCIPLINE

- A service never imports a component or touches the DOM.
- Every method maps the response to a typed interface before returning it.
- Cancellation is the caller's job -- switchMap or takeUntilDestroyed, not the service's.
- No service hardcodes a base URL -- it reads environment.apiUrl.

Injectable service (excerpt)

```
@Injectable({ providedIn: 'root' })
export class InteractionsService {
  constructor(private http: HttpClient) {}
  create(customerId: string, body: CreateInteractionRequest) {
    return this.http.post<InteractionResponse>(
      `${environment.apiUrl}/customers/${customerId}/interactions`, body);
  }
}
```

KEY TAKEAWAY A component that injects HttpClient directly instead of the service is exactly the layering violation this review exists to catch.

ANGULAR SERVICE LAYER

Services encapsulate business logic and data access, promote reusability, and act as a bridge between components and APIs.

KEY RESPONSIBILITIES



BUSINESS LOGIC

Encapsulate and reuse business rules across multiple components.



DATA ACCESS

Communicate with REST APIs using HttpClient and return typed data.



SHARED STATE

Manage and share application state using services and RxJS.



ABSTRACTION

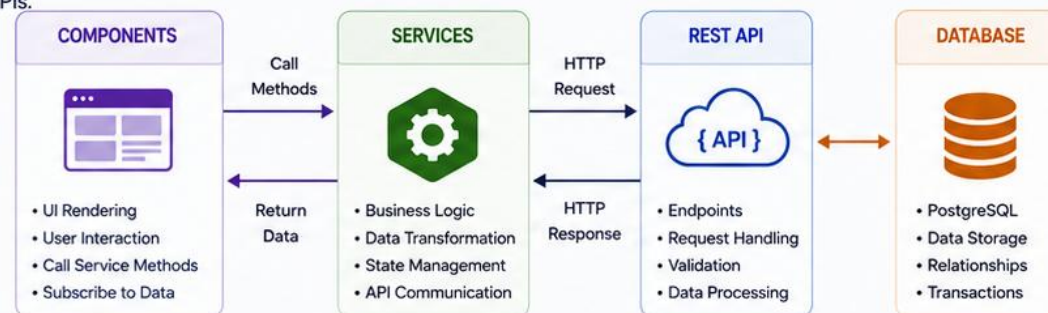
Abstract API details from components for cleaner and testable code.



REUSABILITY

Enable reuse of logic across features and components.

SERVICE LAYER ARCHITECTURE



SERVICE LAYER FLOW



Services keep components lean and focused on presentation.

BEST PRACTICES



SINGLE RESPONSIBILITY

Each service should have a clear and focused purpose.



INJECT, DON'T NEW

Use Angular Dependency Injection for services.



RETURN OBSERVABLES

Return Observables, not subscriptions, from services.



HANDLE ERRORS

Centralize error handling in services.



KEEP SERVICES SMALL

Break large services into smaller, purpose-driven services.



TYPE EVERYTHING

Use TypeScript interfaces for requests and responses.

TYPES OF SERVICES



API SERVICES

Communicate with backend APIs



BUSINESS SERVICES

Encapsulate business logic and rules



DATA SERVICES

Manage client-side or shared data



UTILITY SERVICES

Provide helper functions and tools

BENEFITS



IMPROVED
REUSABILITY



EASIER
MAINTENANCE



BETTER TESTABILITY
& SCALABILITY



CLEANER & MORE
ORGANIZED CODE

REST API CLIENT DESIGN

A quick review of how the Angular client talks to Module 49's Spring Boot API -- typed, centralized, and cancellable.

Term	Meaning
Typed service	One injectable class per resource that knows the base URL and response shapes.
takeUntilDestroyed()	Cancels a subscription automatically when the component is destroyed.
Correlation header	X-Correlation-ID: lab-request-001 sent on every write, matching Module 49.
HttpErrorResponse	Angular's typed wrapper around the 400/404 Problem Details body from Module 49.

WHY IT MATTERS

**One Base URL**

Every service reads environment.apiUrl -- no hardcoded hosts.

**Typed Requests/Responses**

TypeScript interfaces mirror Module 49's DTOs field for field.

**Cancellation-Safe**

takeUntilDestroyed and switchMap stop a slow, superseded request from winning a race.

**Correlation Always Sent**

lab-request-001 on writes -- traceable end to end with Module 49's logs.

KEY TAKEAWAY An ad-hoc HttpClient call copy-pasted into a component drifts from Module 49's contract the moment that contract changes -- typed services are the only source of truth.

REST API CLIENT DESIGN

A well-designed REST API client ensures **maintainability**, **reusability**, and a **consistent** way to communicate with backend services.



Key Principles



Separation of Concerns

Keep HTTP communication separate from components.



Reusability

Centralize API calls in services and avoid duplicate logic.



Maintainability

Use typed models, consistent patterns, and clear error handling.



Scalability

Design for growth with interceptors, config, and modular services.

Layered REST Client Architecture



Component (UI Layer)

Calls service methods and handles UI state.



Service Layer (API Client)

Contains methods for each API endpoint.



HTTP Interceptors

Add headers (Auth), handle errors, logging, etc.



HTTP Client (HttpClient)

Executes HTTP requests and returns Observables.



REST API (Backend)

Spring Boot REST endpoints.

Example (Angular + TypeScript)

```
// api.service.ts
import { Injectable } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Employee } from '../models/employee.model';

@Injectable({ providedIn: 'root' })
export class EmployeeService {
  private baseUrl = environment.apiUrl + '/api/employees';
  constructor(private http: HttpClient) {}

  getEmployees(page: number, size: number): Observable<any> {
    let params = new HttpParams()
      .set('page', page)
      .set('size', size);
    return this.http.get<any>(this.baseUrl, { params });
  }

  getEmployee(id: number): Observable<Employee> {
    return this.http.get<Employee>(`${this.baseUrl}/${id}`);
  }

  createEmployee(emp: Employee): Observable<Employee> {
    return this.http.post<Employee>(this.baseUrl, emp);
  }
}
```

Best Practices



Use a dedicated service per domain or resource.



Return typed models or DTOs from API calls.



Use interceptors for auth, logging, and error handling.



Store API base URL in environment configuration.



Handle errors centrally and show user-friendly messages.



Use Observables and AsyncPipe for UI updates.

DTO ALIGNMENT

TypeScript interfaces mirror Module 49's Java records field for field -- no JPA entity shape ever reaches the browser.

INTERFACE SHAPE

CreateInteractionRequest: channel, summary -- matches the Java record exactly.

InteractionResponse: id, customerId, channel, summary, createdAt.

channel is a union type 'PHONE' | 'EMAIL' | 'CHAT', not an open string.

No interface ever adds a field the backend does not actually send.

CONTRACT RULES

- An additive backend field is safe; a renamed or removed field must fix the interface, not silently widen the type.
- docs/backend-demo.md is the frozen source of truth for every field name.
- A mapper function converts InteractionResponse to the view model, not the template.
- No component reads a raw JSON field the interface does not declare.

TypeScript interface (excerpt)

```
export interface InteractionResponse {  
  id: string; customerId: string;  
  channel: 'PHONE' | 'EMAIL' | 'CHAT';  
  summary: string; createdAt: string;  
}
```

KEY TAKEAWAY A TypeScript interface that drifts from Module 49's actual DTO is exactly the silent contract mismatch this alignment discipline exists to prevent.

TRY IT YOURSELF — EXERCISE 2: PIN THE API CLIENT CONTRACT

Reinforces: the Angular service layer, REST client design, and DTO alignment you just covered — pinned as a typed contract before any HttpClient call is written.

STEPS (notes/lab50-api-client-contract.md)

Step	What To Do
1 — Method Table	List the service methods: list, getById, create, update, delete — with verb and URL.
2 — DTO Fields	Write the CustomerDto fields and their TypeScript types, matching the Java DTO.
3 — Mismatch Risk	Name one field where Java and TypeScript naming or nullability could drift.
4 — Scope	Contract only — no HttpClient code, no interceptor yet.

Service contract sketch

```
list()           GET    /api/customers?page=0&size=20
getById(id)      GET    /api/customers/{id}      -> CustomerDto
create(dto)      POST   /api/customers           -> 201 + Location
```

TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-api-client-contract.md (workspace: examples/module-50-exercises).

DTO ALIGNMENT

DTO (Data Transfer Object) alignment ensures the frontend and backend share the same contract for data structures.



TYPE SCRIPT DTO (Frontend)

```
// employee.model.ts
export interface EmployeeDto {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  departmentId: number;
  salary: number;
  active: boolean;
  createdAt: string;
}
```

JSON OVER THE WIRE

```
{
  "id": 101,
  "firstName": "Jane",
  "lastName": "Doe",
  "email": "jane.doe@company.com",
  "departmentId": 10,
  "salary": 75000.50,
  "active": true,
  "createdAt": "2024-05-15T10:30:00Z"
}
```

JAVA DTO (Backend)

```
// EmployeeDto.java
public class EmployeeDto {
  private Long id;
  private String firstName;
  private String lastName;
  private String email;
  private Long departmentId;
  private BigDecimal salary;
  private Boolean active;
  private OffsetDateTime createdAt;
}
```

ALIGNMENT CHECKLIST

- ✓ Same field names
- ✓ Compatible data types
- ✓ Required vs optional fields
- ✓ Consistent date/time formats
- ✓ Enum values match
- ✓ Version changes managed

KEY TAKEAWAY

DTO alignment creates a stable, reliable, and secure contract between the frontend and backend.

WHY DTO ALIGNMENT MATTERS



Consistent Contract

Both client and server agree on data structure and field names.



Reduces Integration Errors

Prevents runtime issues caused by mismatched fields or types.



Easier Maintenance

Changes are intentional, versioned, and backward compatible.



Improved Security

Expose only required data, avoid leaking internal entities.



Better Performance

Transfer only necessary data over the network.

BEST PRACTICES

- Use DTOs, not entities, in API contracts.
- Keep DTOs simple and purpose-specific.
- Use validation on DTOs.
- Maintain backward compatibility.
- Document DTOs in OpenAPI.

AUTHENTICATION INTEGRATION

The Angular client consumes Module 49's JWT/RBAC stubs -- login is stubbed, but the token path is real.



Token On Login

A stub login screen exchanges credentials for a JWT issued by Module 49's security config.



Token In Memory

The token lives in an injectable AuthService signal, not in a global variable or the DOM.



Guard Protected Routes

canActivate guards redirect to login when no valid session exists.



Role Awareness

AGENT and MANAGER roles from the token gate which actions the UI even offers.

KEY TAKEAWAY A token stored in a plain module-level variable, readable by any script on the page, fails this module's security review before it fails anything else.

AUTHENTICATION INTEGRATION

Integrate secure authentication to protect APIs and ensure that only authorized users can access backend resources.



Authentication Flow

- 1 User logs in via Angular UI (email + password).
- 2 Angular sends credentials to /api/auth/login.
- 3 Spring Boot validates user and returns a JWT token.
- 4 Angular stores the JWT securely (e.g., in memory or localStorage).
- 5 Angular includes the JWT in the Authorization header for all API requests.
- 6 Spring Boot validates the JWT and authorizes the request.

JWT in Request Header

Authorization: Bearer <your-jwt-token>

Benefits

- ✓ **Stateless:** No server-side sessions.
- ✓ **Scalable:** Works across microservices.
- ✓ **Secure:** Signed token prevents tampering.
- ✓ **Flexible:** Supports roles and permissions.

Token Contents (Example)

```
{
  "sub": "user123",
  "email": "jane.doe@company.com",
  "roles": ["ROLE_USER", "ROLE_ADMIN"],
  "iat": 1715775300,
  "exp": 1715778900
}
```

Angular: Login and Token Usage

Login Request (Angular Service)

```
return this.http.post<LoginResponse>(environment.apiUrl + '/api/auth/login', {
  email: this.email,
  password: this.password
}).pipe(
  tap(res => {
    localStorage.setItem('access_token', res.token);
    this.currentUser.set(res.user);
  })
);
```

Attach Token (Http Interceptor)

```
intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
  const token = localStorage.getItem('access_token');
  if (token) {
    const cloned = req.clone({
      headers: { Authorization: `Bearer ${token}` }
    });
    return next.handle(cloned);
  }
  return next.handle(req);
}
```

Best Practices



Use HTTPS for all requests.



Store tokens securely.



Use short-lived tokens.



Implement refresh token strategy.



Validate roles/permissions on the backend.

Security Reminders



- Never expose tokens in URLs.
- Do not store sensitive data in JWT payload.
- Always verify token signature and expiry.

JWT, INTERCEPTORS, ERRORS, AND FORMS

Four disciplines that make every write in this journey safe, traceable, and recoverable.



JWT Handling

Token attaches to every request; a 401 clears the session and redirects to login, never silently retries.



HTTP Interceptors

One functional interceptor attaches the Authorization header and X-Correlation-ID; another maps errors.



Error Handling

catchError maps Module 49's Problem Details to a specific, role=alert message -- never a generic banner.



Form Validation

Reactive Forms validators run client-side for UX; the API still validates -- the browser is never the boundary.

KEY TAKEAWAY A token stored insecurely, an interceptor that swallows errors, or a form that trusts client validation as the boundary are three separate graded deductions.



MODULE 50

Capstone Build:
Frontend and Persistence

JWT Handling

Secure stateless authentication between Angular frontend and Spring Boot backend

WHAT IS JWT?

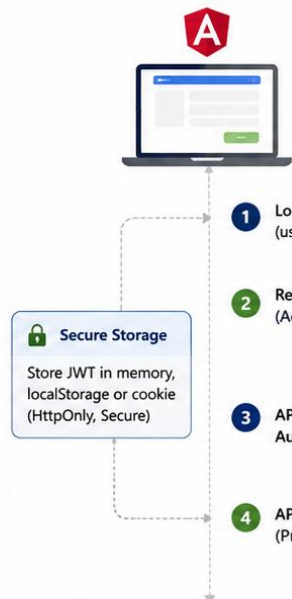
JWT (JSON Web Token) is a compact, URL-safe token used to securely transmit claims between client and server.

- ✓ **Stateless:** No server-side session storage
- ✓ **Self-contained:** Contains claims (user, roles, expiry, etc.)
- ✓ **Secure:** Signed using a secret key or asymmetric key pair

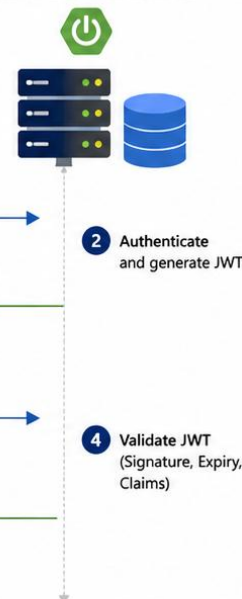
TYPICAL FLOW

- 1 User logs in with username/password
- 2 Backend authenticates and returns JWT
- 3 Client stores JWT securely
- 4 Client includes JWT in Authorization header
- 5 Backend validates token and processes request

ANGULAR FRONTEND



SPRING BOOT BACKEND

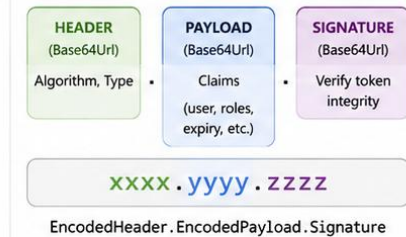


- 1 Login Request (username, password)
- 2 Return JWT Token (Access Token)
- 3 API Request
Authorization: Bearer <JWT>
- 4 API Response (Protected Resources)

BEST PRACTICES

- ✓ Store tokens securely (HttpOnly cookies preferred for web apps)
- ✓ Use short-lived access tokens (e.g., 15 minutes)
- ✓ Use refresh tokens for long-lived sessions
- ✓ Always validate token signature and expiry
- ✓ Never trust data from the client

JWT STRUCTURE



KEY TAKEAWAY: JWT enables secure, stateless authentication. Handle tokens securely in the frontend and validate them on every request to protect your APIs.



MODULE 35

Frontend to API Integration

HTTP Interceptors

Intercept and transform HTTP requests and responses globally.

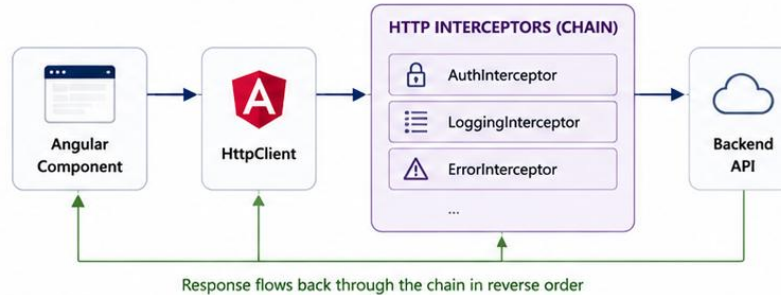
? WHAT IS AN HTTP INTERCEPTOR?

An HTTP Interceptor is a service that sits between your application and the backend API. It can inspect, modify, add headers, handle errors, or transform both requests and responses.

WHY USE INTERCEPTORS?

- ✓ Add auth tokens to every request
- ✓ Handle global errors
- ✓ Log requests and responses
- ✓ Show/hide loading indicators
- ✓ Centralize cross-cutting concerns

HOW INTERCEPTORS WORK



REQUEST FLOW EXAMPLE

- 1 Component sends request using HttpClient
- 2 Request passes through interceptor chain
- 3 Interceptors can modify the request
 - e.g., add Authorization header
- 4 Request reaches the backend API
- 5 Response returns through the chain
 - Interceptors can transform or handle it

IMPLEMENTATION EXAMPLE

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private authService: AuthService) {}

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = this.authService.getToken();
    const cloned = token
      ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
      : req;
    return next.handle(cloned);
  }
}
```



PROVIDING INTERCEPTORS

```
Provide in app.config.ts (Angular 15+)
import { provideHttpClient, withInterceptors }
  from '@angular/common/http';
import { AuthInterceptor } from './interceptors/auth.interceptor';
import { ErrorInterceptor } from './interceptors/error.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([
        AuthInterceptor,
        ErrorInterceptor
      ])
    )
  ]
};
```



COMMON INTERCEPTOR USE CASES

- Authentication**
Attach JWT token to Authorization header
- Error Handling**
Centralize API error handling and user messages
- Logging**
Log requests and responses for debugging
- Loading Indicator**
Show/hide global loader during API calls
- Base URL / Multi-tenancy**
Update base URL or tenant header dynamically



KEY TAKEAWAY: HTTP Interceptors help you apply common logic globally, keep your code clean, and make your Angular applications more secure, maintainable, and enterprise-ready.



MODULE 35

Frontend to API Integration

Error Handling

Handle API errors gracefully and provide meaningful feedback to users.



WHY ERROR HANDLING MATTERS

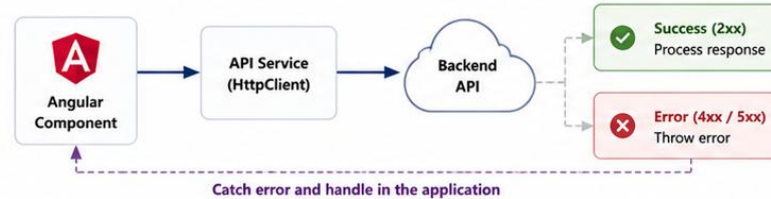
- ✓ Improves user experience
- ✓ Prevents application crashes
- ✓ Provides meaningful feedback
- ✓ Helps with debugging and monitoring
- ✓ Ensures reliability and resilience



COMMON HTTP ERROR CODES

Code	Meaning	When It Occurs
400	Bad Request	Invalid request or validation failure
401	Unauthorized	Authentication required or token invalid
403	Forbidden	User has no access to the resource
404	Not Found	Resource not found
500	Internal Server Error	Unexpected server error
503	Service Unavailable	Server is down or unavailable

ERROR FLOW



EXAMPLE: GLOBAL HTTP ERROR HANDLING SERVICE

```
@Injectable()
export class GlobalErrorHandler {
  handleError(error: HttpErrorResponse): Observable<never> {
    let message = 'An unexpected error occurred';

    if (error.error instanceof ErrorEvent) {
      // Client-side error
      message = `Client Error: ${error.error.message}`;
    } else {
      // Server-side error
      switch (error.status) {
        case 400: message = 'Invalid request. Please check your input.'; break;
        case 401: message = 'Session expired. Please login again.'; break;
        case 403: message = 'You do not have permission to perform this action.'; break;
        case 404: message = 'Resource not found.'; break;
        case 500: message = 'Server error. Please try again later.'; break;
        default: message = `Error ${error.status}: ${error.message}`;
      }
    }

    // Log the error (monitoring / debugging)
    console.error('API Error:', error);

    // Show user-friendly message (toast, alert, etc.)
    return throwError(() => new Error(message));
  }
}
```



BEST PRACTICES

- ⚙️ Centralize error handling in a service or interceptor
- 👤 Provide user-friendly messages
- 📄 Log technical details for debugging
- 🔒 Do not expose sensitive server errors to users
- 🔗 Use consistent error response format from the backend
- 📶 Handle network errors and timeouts



USER EXPERIENCE EXAMPLES

- ✓ **Success**
Employee saved successfully.
- ⚠️ **Validation Error (400)**
Please check the highlighted fields.
- 🔒 **Unauthorized (401)**
Your session has expired. Please login again.
- ✗ **Not Found (404)**
The requested resource was not found.
- 🔴 **Server Error (500)**
Something went wrong. Please try again later.



KEY TAKEAWAY: Good error handling improves user trust and system reliability. Handle errors centrally, log details, and always show helpful messages to users.



MODULE 50

Capstone Build:
Frontend and Persistence

Form Validation

Ensure data quality and provide immediate feedback to users.



WHY FORM VALIDATION?

- ✓ Prevents invalid data from reaching the API
- ✓ Improves user experience with instant feedback
- ✓ Reduces server errors and rework
- ✓ Enforces business rules at the UI layer
- ✓ Supports accessibility and usability



TYPES OF VALIDATION



Template-Driven Forms

- Simple to implement
- Good for basic scenarios



Reactive Forms

- More explicit and scalable
- Best for complex forms

EXAMPLE: VALIDATED USER REGISTRATION FORM

Full Name *
 Looks good!

Email *
 Valid email address

Password *
 Password must be at least 8 characters

Confirm Password *
 Passwords do not match

Age
 You must be at least 18 years old

Create Account

COMMON VALIDATORS (ANGULAR)

- *** required
Field is mandatory
- Aa** minlength / maxlength
Enforce length constraints
- 123** min / max
Enforce numeric range
- *** pattern
Validate using regex
- ✉** email
Validate email format
- 🔧** custom validator
Reusable business rules



VALIDATION FEEDBACK STRATEGIES

- ⚡** Instant Feedback
Show errors as the user types
- 👆** On Touch / On Blur
Show errors after field is touched
- 🚀** On Submit
Validate all fields on form submit
- 🗿** Accessible Messages
Use clear, screen-reader friendly text



BEST PRACTICES

- ✓ Validate on both client and server
- ✓ Provide clear and actionable messages
- ✓ Highlight invalid fields
- ✓ Prevent form submission when invalid
- ✓ Keep validation rules consistent with backend
- ✓ Use custom validators for business rules

REACTIVE FORM VALIDATION EXAMPLE

```
this.registerForm = this.fb.group({
  fullName: ['', [Validators.required]],
  email: ['', [Validators.required, Validators.email]],
  password: ['', [Validators.required, Validators.minLength(8)]],
  confirmPassword: ['', Validators.required],
  age: [null, [Validators.required, Validators.min(18)]]
}, { validators: this.passwordMatchValidator });
```

CUSTOM VALIDATOR EXAMPLE (PASSWORD MATCH)

```
passwordMatchValidator(form: AbstractGroup): ValidationErrors | null {
  const password = form.get('password')?.value;
  const confirm = form.get('confirmPassword')?.value;
  return password === confirm ? null : { mismatch: true };
}
```

HTML ERROR DISPLAY EXAMPLE

```
<input formControlName="email" type="email" />
<div *ngIf="email?.invalid && email?.touched" class="error">
  <div *ngIf="email?.errors?.['required']">Email is required</div>
  <div *ngIf="email?.errors?.['email']">Enter a valid email</div>
</div>
```



KEY TAKEAWAY: Good form validation ensures data integrity, improves user experience, and reduces backend errors.

TRY IT YOURSELF — EXERCISE 3: PLAN JWT AND ERROR STATES

Reinforces: the JWT interceptor, error handling, and form validation slides you just covered, planned as concrete UI states for the customer slice.

STEPS (notes/lab50-jwt-error-states.md)

Step	What To Do
1 — Interceptor Rule	Say which requests get the Authorization header and which are exempt.
2 — Status Map	Map 401, 403, 404, and 409 each to one specific UI behaviour.
3 — Empty vs Loading	Distinguish the loading state from the genuinely-empty result state.
4 — Scope	Plan only — no interceptor code and no real identity provider.

Status → UI behaviour

```
401 -> clear token, route to /login    403 -> inline 'not permitted'
404 -> empty-state card for CUS-9999  409 -> keep form, show conflict
loading != empty: spinner while pending, card only after a 200
```

TRY IT NOW — Complete Exercise 3 before continuing: `exercises/exercise-03-jwt-and-error-states.md` (workspace: `examples/module-50-exercises`).

LOADING AND EMPTY STATES

Six states, six behaviors -- happy-path-only UIs lie to the agent the moment PostgreSQL or the API has a bad day.

State	Trigger	UI Behavior
Loading	In-flight GET/POST	Spinner or skeleton; submit disabled.
Empty	No interactions yet	Explicit empty message, not a blank panel.
Success	201 response	Timeline refetch; brief confirmation.
Invalid	400 response	role="alert" field errors.
Unauthorized	401/403 response	Sign-in or access message.
Outage	Network error or 5xx	Retry guidance -- never a fake success.

WHY IT MATTERS

1 Disable, Don't Double-Post

One in-flight request at a time -
- a double click makes one row.

2 Outage Is Not Silence

A stopped API shows a real
outage message, never a blank
page.

3 Refresh After Write

The timeline refetches; it is
never hand-patched
optimistically.

4 Test With Backend Stopped

One deliberate outage test
captures real evidence, not a
guess.

KEY TAKEAWAY Test with the backend stopped once, on purpose -- capturing real outage UI evidence beats assuming it 'probably works'.



MODULE 50

Capstone Build:
Frontend and Persistence

Loading and Empty States

Keep users informed while data is being fetched or when no data is available.



LOADING STATES



Loading states indicate that the system is working. They prevent confusion and improve user experience.

EXAMPLES

Spinner



Loading...

Skeleton Screen



Progress Bar

Loading data...
65%



When to Use

- When initial data is being fetched
- When user triggers an action (e.g., search, filter, save)
- For longer operations, prefer progress indicator or skeleton



EMPTY STATES



Empty states inform users that no data is available. They guide users on what to do next.

EXAMPLES

No Results Found



No results found

We couldn't find any matching items for your search.

Clear Search

No Data Yet



No data available

You haven't added any items yet.

Add New Item

Filtered Result Empty



No items match your filters

Try adjusting or clearing your filters.

Clear Filters



When to Use

- When API returns an empty array / null
- When filters or search yield no results
- When user has not created any data yet



BEST PRACTICES

- ✓ Always show feedback for user actions
- ✓ Use appropriate indicators based on operation duration
- ✓ Avoid flicker – use minimal loading time thresholds
- ✓ Skeleton screens improve perceived performance
- ✓ Provide helpful messages and clear next steps
- ✓ Keep design consistent across the application
- ✓ Ensure accessibility (ARIA roles, color contrast)

ACCESSIBILITY TIPS



- Use `aria-busy="true"` during loading
- Use `aria-live="polite"` to announce state changes



KEY TAKEAWAY: Good loading and empty states reduce uncertainty, improve usability, and guide users toward their next action.



Keep users informed



Reduce confusion



Improve user experience

POSTGRESQL SCHEMA REVIEW

customer_interaction already exists from Module 49 -- this module reviews it against PostgreSQL-specific types and constraints.

SCHEMA ELEMENTS

interaction_id, customer_id, channel, summary, created_at, version.

Primary key on interaction_id; foreign key to customer.

CHECK constraint restricts channel to PHONE/EMAIL/CHAT.

Index on (customer_id, created_at) for the timeline query.

REVIEW DISCIPLINE

- Confirm every type is genuinely PostgreSQL-native, not assumed from habit.
- Named constraints -- pk_/fk_/ck_ prefixes, never anonymous.
- Migration applies cleanly on the shared training PostgreSQL instance.
- Verify the generated SQL once in a test or log -- never just assume it.

Migration excerpt (from the lab guide)

```
CREATE TABLE customer_interaction (  
  interaction_id UUID NOT NULL,  
  customer_id UUID NOT NULL,  
  channel VARCHAR(20) NOT NULL,  
  CONSTRAINT ck_interaction_channel CHECK (channel IN ('PHONE','EMAIL','CHAT'))  
);
```

KEY TAKEAWAY A migration copied without adapting types is a named failure mode -- verify every column against PostgreSQL specifically.



MODULE 50

Capstone Build:
Frontend and Persistence

PostgreSQL Schema Review

Relational schema for the Capstone application (simplified overview).

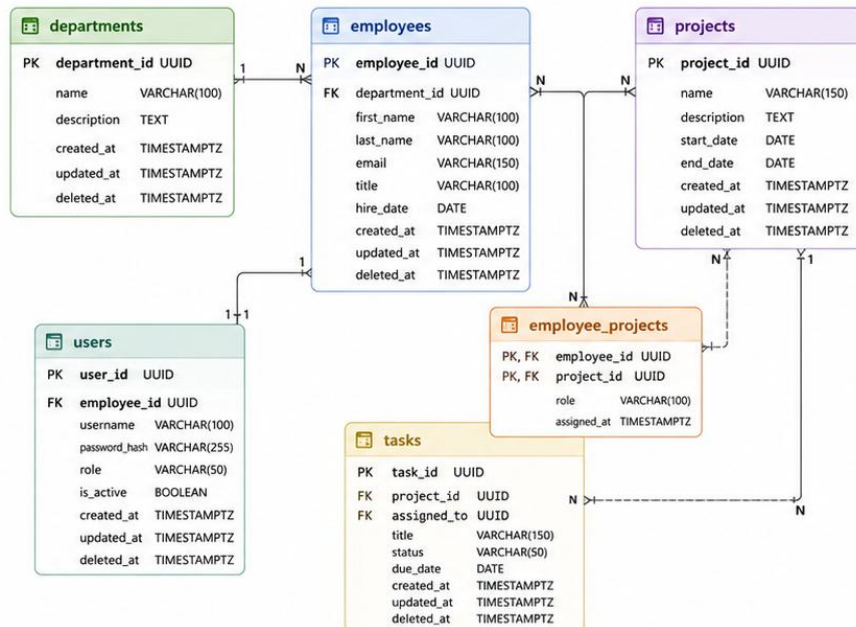
SCHEMA OVERVIEW

- Primary entities for the Capstone application
- Normalized for data integrity and consistency
- Uses UUID primary keys
- Audit columns on all core tables
- Soft delete support with `deleted_at`
- Timestamps in UTC

NAMING CONVENTIONS

Tables	Lowercase, plural (e.g., employees)
Columns	Lowercase snake_case (e.g., first_name)
PK	<table>_id (UUID)
FK	<table>_id
Timestamps	created_at, updated_at
Soft Delete	deleted_at (nullable)

ENTITY RELATIONSHIP DIAGRAM



KEY RELATIONSHIPS

- departments 1 — N employees**
Each department has many employees
- employees N — N projects**
Employees work on many projects and projects have many employees
- projects 1 — N tasks**
Each project has many tasks
- employees 1 — 1 users**
Each employee has one user account

INDEXES (EXAMPLES)

- `CREATE INDEX idx_employees_dept ON employees (department_id);`
- `CREATE INDEX idx_projects_dates ON projects (start_date, end_date);`
- `CREATE INDEX idx_tasks_project ON tasks (project_id);`
- `CREATE INDEX idx_tasks_assigned ON tasks (assigned_to);`



DESIGN PRINCIPLES

- ✓ Referential integrity with foreign keys
- ✓ Minimal redundancy
- ✓ Auditability with timestamps
- ✓ Soft delete for recoverability



POSTGRESQL FEATURES USED

- ✓ UUID primary keys
- ✓ TIMESTAMPTZ for time tracking
- ✓ Foreign Keys with ON DELETE RESTRICT
- ✓ Indexes for performance



KEY TAKEAWAY

A well-designed schema ensures data integrity, scalability, and performance. This schema supports the core business operations of the Capstone application efficiently.



ENTITY-TO-TABLE MAPPING

JPA entity fields must map exactly to migration columns -- confirmed by a real repository test, not assumed by reading the class.

MAPPING CHECKLIST

Every entity field maps to an actual migration column, one to one.

@Version on the version column enables optimistic locking.

Lazy fetch preferred; an explicit timeline query avoids N+1.

No cascade type applied without a written reason.

VERIFY BY TEST

- Run a repository test loading Amina's interactions for real.
- LazyInitializationException means a transactional-boundary problem, not a config toggle to hide it.
- Confirm no accidental cascade delete of Customer.
- Record mapping notes in the demo doc, not tribal knowledge.

Entity mapping (concept)

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "customer_id")
private Customer customer;
@Version private int version;
```

KEY TAKEAWAY Open Session in View used silently as a default, with no ADR behind it, is explicitly flagged rather than accepted as a fix for LazyInitializationException.



MODULE 50

Capstone Build:
Frontend and Persistence

Entity-to-Table Mapping

JPA entities are mapped to relational database tables using annotations and conventions.

WHAT IS ENTITY-TO-TABLE MAPPING?

Object-relational mapping (ORM) maps Java entities to database tables and their relationships.

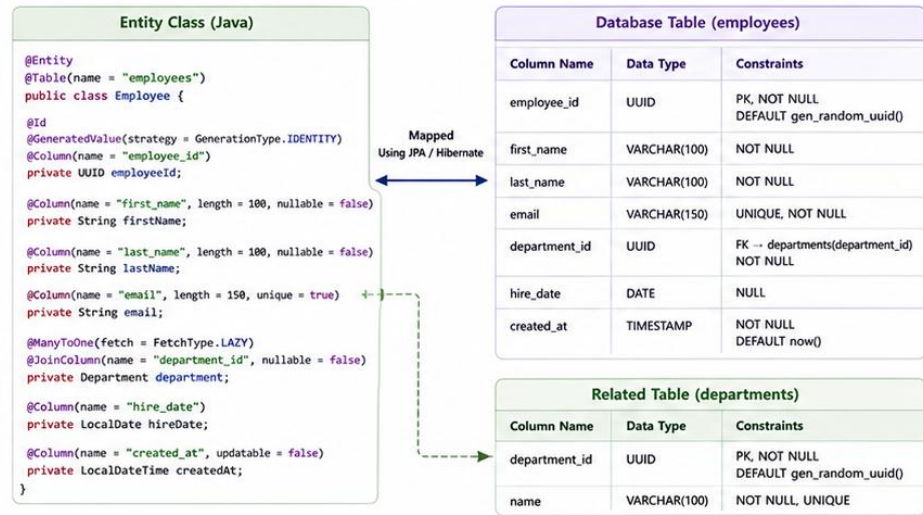
BENEFITS

- ✓ Developer productivity
- ✓ Type-safe data access
- ✓ Database independence
- ✓ Reduced boilerplate SQL
- ✓ Easier maintenance

COMMON ID GENERATION STRATEGIES

- IDENTITY**
Auto-increment / identity column
- SEQUENCE**
Uses database sequence
- TABLE**
Uses a table to generate IDs
- UUID**
Application generates UUIDs

EXAMPLE: EMPLOYEE ENTITY MAPPED TO EMPLOYEES TABLE



MAPPING PRINCIPLES

- One Entity = One Table**
Each entity maps to a single database table.
- Primary Key Mapping**
Every entity must have an identifier (@Id).
- Relationships**
Use annotations to map foreign keys and joins.
- Naming Strategy**
Follow consistent naming for tables and columns.
- Data Integrity**
Let the database enforce constraints and rules.

COMMON ANNOTATIONS

- @Entity**
Marks the class as a JPA entity
- @Table(name = "...")**
Maps entity to a specific table
- @Id**
Defines the primary key field
- @GeneratedValue**
Specifies ID generation strategy
- @Column**
Maps field to table column
- @ManyToOne**
Defines many-to-one relationship
- @JoinColumn**
Specifies foreign key column

BEST PRACTICES

- ✓ Use meaningful table and column names
- ✓ Keep entities focused and cohesive
- ✓ Use correct data types and constraints
- ✓ Prefer LAZY loading for relationships
- ✓ Use @Column to control schema details
- ✓ Review generated schema regularly
- ✓ Enable auditing fields (created_at, updated_at)



KEY TAKEAWAY: Entity-to-table mapping connects your Java domain model with the PostgreSQL schema. Use annotations intentionally, follow best practices, and keep your model aligned with the database.



SPRING DATA REPOSITORY IMPLEMENTATION

Spring Data JPA repositories serve exactly the queries this journey needs -- named by business intent, not generic CRUD guessing.

REPOSITORY METHODS

findByCustomerIdOrderByCreatedAtDesc for the timeline read.

Customer lookup by id, reused across search and profile.

Repository stays persistence-only -- no HTTP or DTO concerns.

Seed Amina and Ravi deterministically for repository tests.

INTEGRATION DISCIPLINE

- Repository methods are named for the business need they serve.
- No repository method returns an entity directly to a controller.
- Avoid N+1 by matching the query shape to the actual UI access pattern.
- Repository tests run against the real training PostgreSQL, not a guess.

Repository method (excerpt)

```
List<Interaction> findByCustomerIdOrderByCreatedAtDesc(UUID customerId);
```

KEY TAKEAWAY A repository test that never actually runs against PostgreSQL only proves the method compiles -- not that the query is correct.



MODULE 50

Capstone Build:
Frontend and Persistence

Spring Data Repository Implementation

Spring Data JPA repositories provide powerful data access with minimal boilerplate code.

WHY REPOSITORIES?

- ✓ Reduces boilerplate CRUD code
- ✓ Built-in query derivation
- ✓ Type-safe and easy to test
- ✓ Integrates with JPA/Hibernate
- ✓ Supports pagination and sorting
- ✓ Improves developer productivity

ENTITY EXAMPLE

```
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private UUID employeeId;

    @Column(name = "first_name", length = 100)
    private String firstName;

    @Column(name = "last_name", length = 100)
    private String lastName;

    @Column(unique = true, length = 150)
    private String email;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
    // getters and setters
}
```

HOW IT WORKS



REPOSITORY INTERFACE

```
// Repository for Employee entity
public interface EmployeeRepository
    extends JpaRepository<Employee, UUID> {

    // Derived query methods
    List<Employee> findByLastName(String lastName);
    Optional<Employee> findByEmail(String email);
    List<Employee> findByDepartment_DeptId(UUID departmentId);
    List<Employee> findByFirstNameContainingIgnoreCase(String firstName);

    // Multiple criteria
    List<Employee> findByDepartment_DeptIdAndLastName(UUID deptId, String lastName);

    // Custom JPQL query
    @Query("SELECT e FROM Employee e WHERE e.hireDate >= :date")
    List<Employee> findHiredAfter(@Param("date") LocalDate date);

    // Native PostgreSQL query
    @Query(value = "SELECT * FROM employees WHERE created_at >= NOW() - INTERVAL '30 days'",
            nativeQuery = true)
    List<Employee> findRecentEmployeesNative();
}
```

USAGE EXAMPLE (SERVICE LAYER)

```
@Service
public class EmployeeService {
    private final EmployeeRepository employeeRepository;

    public EmployeeService(EmployeeRepository employeeRepository) {
        this.employeeRepository = employeeRepository;
    }

    public List<Employee> getEmployeesByDepartment(UUID deptId) {
        return employeeRepository.findByDepartment_DeptId(deptId);
    }
}
```

CORE REPOSITORY INTERFACES

Interface	Purpose
CrudRepository<T, ID>	Basic CRUD operations
PagingAndSortingRepository<T, ID>	Adds pagination and sorting
JpaRepository<T, ID>	Full JPA functionality + CRUD + pagination + sorting
JpaSpecificationExecutor<T>	Support for dynamic queries with Specifications

COMMON DERIVED QUERIES

findBy...	Find by field
findAllBy...	Find multiple by field
findBy...And...	Find by multiple fields
findBy...Or...	Find by multiple fields (OR)
findBy...OrderBy...	Find and sort
findBy...Containing	LIKE %value%
findBy...Between	Range query
findTop / findFirstBy...	Limit results

BEST PRACTICES

- ✓ Extend JpaRepository for most use cases
- ✓ Use meaningful method names
- ✓ Prefer derived queries before @Query
- ✓ Use @Query for complex queries
- ✓ Use pagination and sorting for large datasets
- ✓ Keep repository interfaces small and focused
- ✓ Write integration tests for repositories



KEY TAKEAWAY: Spring Data repositories simplify data access by providing powerful abstractions, query derivation, and integration with JPA/Hibernate. This leads to cleaner code and faster development.



DATABASE RELATIONSHIPS

Customer owns its interactions -- get relationship ownership and cardinality right before writing a single JPA annotation.

RELATIONSHIP SHAPE

One customer has many interactions (one-to-many).

Interaction is the owning side -- it holds the foreign key.

Customer never cascades a delete onto historical interactions by default.

The relationship direction matches the timeline query's access pattern.

REVIEW DISCIPLINE

- Confirm cardinality on paper before annotating an entity.
- Document ownership explicitly -- do not leave it implicit.
- A missing foreign key means an orphaned interaction row is possible.
- Relationship choices must match whatever Module 48's ADR decided.

Relationship note (concept)

```
// Interaction owns the FK to Customer (interaction.customer_id)
// Customer does NOT hold a @OneToMany collection unless the
// timeline query specifically needs it loaded that way
```

KEY TAKEAWAY An unexamined cascade delete on Customer is exactly the kind of accidental data loss this review exists to catch before it reaches production code.

MODULE 39

Spring Data JPA and
PostgreSQL Integration

Database Relationships

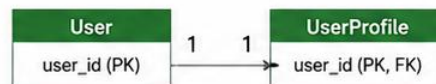
Relationships define how tables are connected. They enforce data integrity and enable meaningful queries across related data.

TYPES OF RELATIONSHIPS



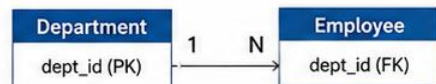
One-to-One (1:1)

Each record in Table A is associated with one record in Table B, and vice versa.



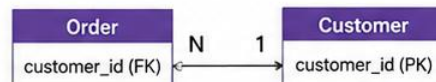
One-to-Many (1:N)

One record in Table A can be associated with many records in Table B.



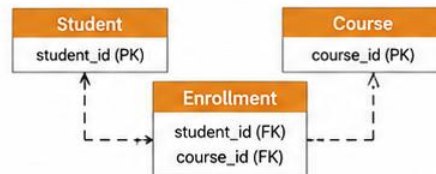
Many-to-One (N:1)

Many records in Table A are associated with one record in Table B.

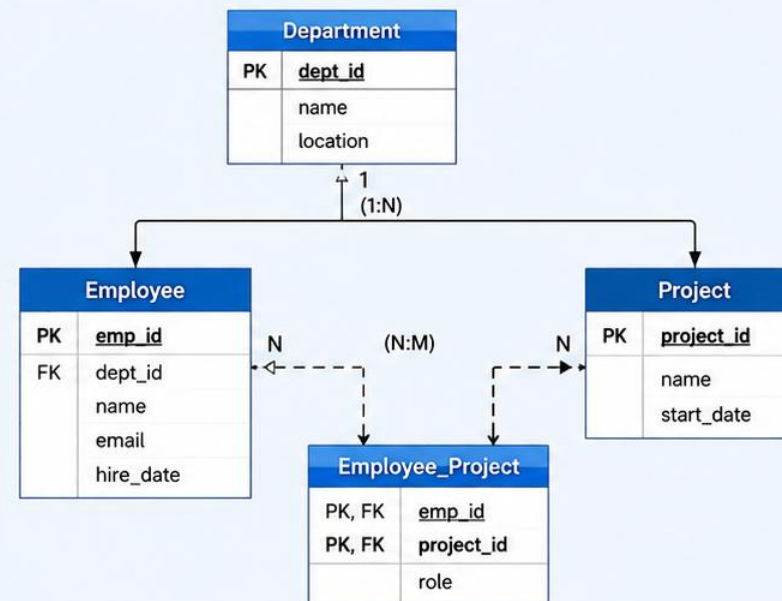


Many-to-Many (N:M)

Many records in Table A are associated with many records in Table B. Requires a junction table.



EXAMPLE: EMPLOYEE MANAGEMENT



KEY TAKEAWAYS



Relationships enforce data integrity using primary and foreign keys.



They enable complex queries and meaningful data retrieval.



Choose the right relationship type based on business rules.



Well-designed relationships improve performance and maintainability.

TRANSACTION BOUNDARIES

The service layer owns every transaction -- a repository or controller that opens its own is a boundary violation.

BOUNDARY RULES

@Transactional sits on the service method, never the repository or controller.

A single interaction create is one atomic unit -- save succeeds or nothing does.

Reads that only need one query can skip an explicit transaction; writes never do.

The transaction commits before any downstream Kafka event publishes.

Transactional service (concept)

```
@Transactional
public InteractionResponse create(UUID customerId, CreateInteractionRequest req) {
    Customer c = customers.findById(customerId).orElseThrow(...);
    Interaction saved = interactions.save(new Interaction(c, req));
    return mapper.toResponse(saved);
}
```

FAILURE DISCIPLINE

- A validation failure must roll back before touching the database, not after.
- An exception inside the service must propagate, not be swallowed and logged only.
- readOnly = true on pure read methods documents intent for the query planner.
- A LazyInitializationException outside the boundary means the boundary is drawn wrong.

KEY TAKEAWAY A row that exists in the UI but not in PostgreSQL after a restart almost always traces back to a transaction boundary drawn in the wrong place.

MODULE 50

Capstone Build:
Frontend and Persistence

Transaction Boundaries

A transaction boundary defines the start and end of a unit of work.

All operations within the boundary either complete successfully (COMMIT) or are fully rolled back (ROLLBACK).

WHAT IS A TRANSACTION?

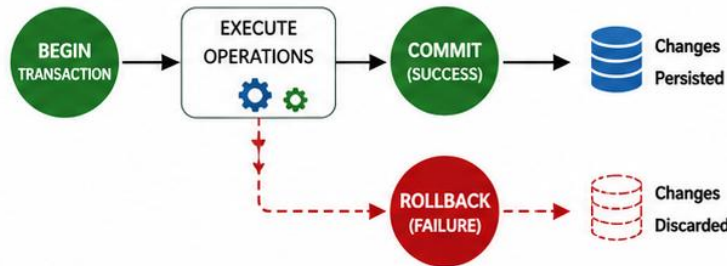


A transaction is a sequence of one or more database operations treated as a single logical unit of work.

ACID Guarantee

Atomicity, Consistency, Isolation, Durability

TRANSACTION LIFECYCLE



DEFINING BOUNDARIES

- ✓ Start a transaction when a **business operation** begins.
- ✓ End the transaction only when the operation is **complete**.
- ✓ Include all **dependent operations** within the same boundary.
- ✓ Do not cross boundaries for **unrelated operations**.
- ✓ Keep transactions **short** to reduce locks and contention.

EXAMPLES OF TRANSACTION BOUNDARIES

Business Operation	Includes	Boundary	Outcome
Place Order	- Validate order - Insert order - Update inventory - Record payment	Single Transaction	✓ COMMIT Order placed and inventory updated
Bank Transfer	- Debit account A - Credit account B - Record transfer	Single Transaction	✓ COMMIT Funds moved safely
Update Profile	- Update user info - Update preferences	Single Transaction	✓ COMMIT All updates saved
Send Email (Async)	Save email log	Separate Transaction	✓ COMMIT Log saved independently

SPRING EXAMPLE

```
@Service
public class OrderService {

    @Transactional
    public void placeOrder(OrderDto dto) {
        validateOrder(dto);
        Order order = orderRepository.save(dto.toEntity());
        inventoryService.updateStock(dto.getItems());
        paymentService.charge(dto.getPayment());
        auditService.record(order.getId(), "ORDER_PLACED");
    }
}
```

i If any exception occurs, Spring marks the transaction for **rollback** and no changes are persisted.

BEST PRACTICES

- Align boundaries with **business operations**.
- Keep transactions **short** and focused.
- Avoid **long-running** transactions.
- Do not include **user think time** in transactions.
- Handle **exceptions** and roll back gracefully.



KEY TAKEAWAY

A well-defined transaction boundary ensures data integrity, consistency, and system reliability.

One business operation = One transaction boundary.

DATABASE MIGRATIONS

Every schema change ships as a versioned, repeatable Flyway migration -- never a hand-edited table on the shared instance.

MIGRATION DISCIPLINE

Numbered SQL files under db/migration/, applied in order, never reordered.

A migration already applied is never edited -- a fix ships as a new migration.

Flyway's schema history table is the source of truth for what has run.

CI runs every migration against a fresh PostgreSQL instance before merge.

COMMON FAILURE MODES

- A migration copied from a different example without adapting its column types.
- Two developers both claiming the same version number -- fixed by rebasing, never renumbering silently.
- A migration that depends on data seeded outside of a migration file.
- Dropping a column that code not yet redeployed still reads.

Migration filename convention

```
V12__add_interaction_channel_check_constraint.sql
```

KEY TAKEAWAY Hibernate 'figuring out' the schema automatically is explicitly discouraged this module -- Flyway migrations are the only accepted source of schema truth.

MODULE 50

Capstone Build:
Frontend and Persistence

Database Migrations

Database migrations are version-controlled changes to the database schema.
They ensure consistent, repeatable, and safe evolution of your database across all environments.

WHY MIGRATIONS MATTER



Consistency

Every environment has the same schema structure and data rules.



Repeatability

Migrations can be applied from scratch on any environment.



Version Control

Schema changes are tracked in source control with your application code.



Team Collaboration

Multiple developers can make changes safely without manual coordination.



Auditability

History of schema changes is preserved with timestamps and authors.

MIGRATION TOOLS



Flyway

- Simple, convention-over-configuration
- SQL-based migrations
- Versioned and repeatable migrations
- Works great with Spring Boot



Liquibase

- Powerful and extensible
- SQL, YAML, JSON, XML formats
- Preconditions and rollback support
- Ideal for complex enterprise schemas

MIGRATION WORKFLOW



1. Create Migration

Developer creates a migration script for the change.



2. Commit

Migration file is committed to version control.



3. Deploy

Application is deployed to the target environment.



4. Apply

Migrations are executed in order to update the schema.



5. Validate

Schema is validated and application starts successfully.



Migrations run automatically at application startup (or via CI/CD) and are applied in version order.

EXAMPLE MIGRATION

V2__create_employee_table.sql

```
CREATE TABLE employee (  
  id SERIAL PRIMARY KEY,  
  first_name VARCHAR(100) NOT NULL,  
  last_name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE,  
  hire_date DATE NOT NULL DEFAULT CURRENT_DATE,  
  department_id INT,  
  CONSTRAINT fk_employee_department  
    FOREIGN KEY (department_id) REFERENCES department(id)  
);
```



V3__add_phone_to_employee.sql

```
ALTER TABLE employee  
ADD COLUMN phone VARCHAR(20);  
  
CREATE INDEX idx_employee_phone  
ON employee(phone);
```

SCHEMA EVOLUTION

Before (V2)

```
id (PK)  
first_name  
last_name  
email  
hire_date  
department_id (FK)
```



After (V3)

```
id (PK)  
first_name  
last_name  
email  
phone  
hire_date  
department_id (FK)
```



BEST PRACTICES



Write small, focused migrations.



Test migrations on a local and test database.



Never edit executed migrations.



Provide rollback scripts where possible.



Review and version-control all migration scripts.



KEY TAKEAWAY

Migrations make database changes predictable, traceable, and safe across all environments and team members.

SEED DATA

Amina and Ravi must exist the same way on every environment -- seeded deterministically, never assumed to be already there.

SEEDING APPROACH

A dedicated Flyway migration (or repeatable R__ script) inserts the fixed fixtures.

CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) get fixed UUIDs.

CUS-9999 is deliberately never seeded -- its absence is the not-found test.

Seed data is idempotent -- re-running the seed script never duplicates rows.

Seed migration (concept)

```
INSERT INTO customer (customer_id, name, status) VALUES
  ('11111111-1111-1111-1111-111111111111', 'Amina Khan', 'ACTIVE'),
  ('22222222-2222-2222-2222-222222222222', 'Ravi Singh', 'PROSPECT');
```

SEEDING DISCIPLINE

- Seed data never contains real customer PII -- fixtures are synthetic by design.
- Integration tests seed their own data in setup, not relying on shared seed rows.
- A seed script is reviewed the same way as any other migration -- named constraints apply.
- Seed timestamps use a fixed value so timeline ordering tests are deterministic.

KEY TAKEAWAY A demo that only works because someone manually inserted a row last week is not reproducible -- seed data must be scripted and versioned.

TRY IT YOURSELF — EXERCISE 4: SKETCH SCHEMA AND MIGRATIONS

Reinforces: the PostgreSQL schema, entity mapping, and Flyway migration slides you just covered, sketched for the capstone slice before any SQL runs.

STEPS (notes/lab50-schema-migrations.md)

Step	What To Do
1 — Tables	Name the tables and their primary keys for the customer and interaction slice.
2 — Migration Order	List the Flyway files in order, with the versioned naming convention.
3 — Index Decision	Name one index and say which query it serves.
4 — Seed Rule	State that seed data is synthetic only: CUS-1001 and CUS-1002.

Flyway migration order

```
V1__create_customer_table.sql      (id uuid pk, status, created_at)
V2__create_interaction_table.sql   (fk customer_id -> customer.id)
V3__index_interaction_customer_occurred_at.sql
```

TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-schema-and-migrations.md (workspace: examples/module-50-exercises).

MODULE 50

Capstone Build:
Frontend and Persistence

Seed Data

Seed data is initial data inserted into the database to provide a known starting state for development, testing, demos, and reference data.

**Consistent Baseline**
Every environment starts with the same data.

**Faster Onboarding**
Developers and testers can be productive immediately.

**Reliable Testing**
Provides known data for repeatable and predictable tests.

**Demo Ready**
Sample data makes applications usable for demos and reviews.

Type	Examples
 Reference Data	Roles, permissions, countries, departments, statuses
 Lookup Data	Categories, priorities, types, tags, configuration options
 Sample Entities	Users, employees, projects, customers, orders
 System Data	Default settings, feature flags, system parameters

**SQL Scripts**
Simple and portable. Great for small to medium datasets.

**Data Migrations**
Versioned seed data using Flyway or Liquibase for full traceability.

**Application Initializers**
Use Spring Boot StartupRunner or CommandLineRunner for dynamic seeds.

**Test Data Builders**
Programmatic seed data in tests using builders or factories.

id	first_name	last_name	email	department_id	role_id	hire_date
1	Alice	Johnson	alice.johnson@example.com	1	2	2024-01-10
2	Bob	Smith	bob.smith@example.com	1	3	2024-01-15
3	Carol	Williams	carol.williams@example.com	2	2	2024-02-01
4	David	Brown	david.brown@example.com	3	1	2024-02-12
5	Eve	Davis	eve.davis@example.com	2	3	2024-02-20

EXAMPLE SQL SEED SCRIPT

```
-- Departments
INSERT INTO department (id, name) VALUES
(1, 'Engineering'), (2, 'HR'), (3, 'Finance');

-- Roles
INSERT INTO role (id, name) VALUES
(1, 'Admin'), (2, 'Manager'), (3, 'Employee');

-- Employees
INSERT INTO employee (id, first_name, last_name, email,
department_id, role_id, hire_date) VALUES
(1, 'Alice', 'Johnson', 'alice.johnson@example.com', 1, 2, '2024-01-10'),
(2, 'Bob', 'Smith', 'bob.smith@example.com', 1, 3, '2024-01-15');
COMMIT;
```

BEST PRACTICES

- ✓ Keep seed data minimal and purposeful.
- ✓ Make seeds idempotent (safe to run multiple times).
- ✓ Do not hardcode environment-specific values.
- ✓ Version seed data with your schema changes.
- ✓ Use realistic but non-sensitive data.
- ✓ Reset/refresh seed data in non-production environments.

KEY TAKEAWAY

Good seed data creates a consistent, reliable, and productive foundation across development, testing, and demo environments.

© 2026 by Innovation In Software Corporation

52

PAGINATION AND SORTING

Amina's timeline is small today, but the repository query is built to scale -- Pageable and Sort, not a full table scan.

SPRING DATA PATTERN

Pageable carries page number, size, and sort into the repository method.

`Page<Interaction> findById(UUID id, Pageable pageable).`

`Sort.by("createdAt").descending()` matches the timeline's newest-first order.

The controller maps page metadata (total, hasNext) into the response DTO.

Paged repository method (excerpt)

```
Page<Interaction> findById(UUID customerId, Pageable pageable);
```

ANGULAR CONSUMPTION

- The typed service passes page/size query params, never fetches an unbounded list.
- InteractionTimeline requests the next page on scroll or an explicit 'load more' action.
- Sort order is fixed to newest-first in this journey -- no client-side re-sort of a partial page.
- A page-size default (e.g., 20) is documented so a peer can reproduce the exact count.

KEY TAKEAWAY Fetching 'all interactions' with no page bound works fine in a demo with two rows and fails silently the day a real customer has two thousand.

MODULE 50

Capstone Build:
Frontend and Persistence

Pagination and Sorting

Pagination limits the number of rows returned per request. Sorting defines the order of results. Together, they improve performance, user experience, and scalability.

WHY IT MATTERS

- Performance**
Avoids returning large datasets that slow down the application.
- Better User Experience**
Users navigate through data in manageable chunks.
- Scalability**
Reduces memory usage and database load on large tables.
- Consistency**
Ensures predictable data access across requests.

PAGINATION OVERVIEW

Always use a stable sort (e.g., by id) to ensure consistent results across pages.

SORTING OVERVIEW

id	name	department	salary
Sorted by name ASC			
1	Alice	Engineering	80000
2	Bob	Finance	70000
3	Carol	HR	60000
4	David	Engineering	90000
...	...	...	...
Sorted by salary DESC			
4	David	Engineering	90000
1	Alice	Engineering	80000
2	Bob	Finance	70000
3	Carol	HR	60000
...	...	...	...

A-Z ↑
High → Low ↓

EXAMPLE REST API

GET /api/employees?page=2&size=20&sort=salary,desc&sort=name,asc

page : Page number (0-based index)
size : Number of records per page
sort : One or more sort fields (field,direction)
direction : asc (ascending) or desc (descending)

```
{
  "content": [ { "id": 4, "name": "David", "salary": 90000 }, ... ],
  "page": 2,
  "size": 20,
  "totalElements": 1000,
  "totalPages": 50,
  "sort": [ { "field": "salary", "direction": "desc" }, { "field": "name", "direction": "asc" } ]
}
```

Response Meta

- content : Current page data
- page : Current page number
- size : Page size
- totalElements : Total records
- totalPages : Total pages
- sort : Applied sorting

SPRING DATA JPA EXAMPLE

```
// Controller
@GetMapping("/employees")
public Page<Employee> getEmployees(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(defaultValue = "id,asc") String sort) {
    Pageable pageable = PageRequest.of(page, size, Sort.by(sort));
    return employeeRepository.findAll(pageable);
}

// Repository
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {
    // Paging and sorting handled by Spring Data JPA
}
```

BEST PRACTICES

- ✓ Always paginate large datasets.
- ✓ Use appropriate page size (e.g., 10–50).
- ✓ Use default sorting (e.g., by id) for consistency.
- ✓ Allow sorting only on indexed columns.
- ✓ Validate and whitelist sortable fields.
- ✓ Avoid OFFSET on very large page numbers (prefer keyset/cursor pagination for deep paging).



KEY TAKEAWAY

Pagination and sorting enable efficient, predictable, and scalable data retrieval for any application.



QUERY PERFORMANCE

The timeline index exists for a reason -- confirm the query planner actually uses it, don't just assume.

WHAT TO VERIFY

EXPLAIN ANALYZE the timeline query -- confirm an index scan, not a sequential scan.

The (customer_id, created_at) composite index matches the query's WHERE and ORDER BY.

N+1 queries from an eager-loaded collection are a performance bug, not just style.

spring.jpa.show-sql (or a query logger) run once during development, not left on in production.

EXPLAIN ANALYZE (concept)

```
EXPLAIN ANALYZE
SELECT * FROM customer_interaction
WHERE customer_id = '11111111-1111-1111-1111-111111111111'
ORDER BY created_at DESC;
```

PERFORMANCE DISCIPLINE

- A slow query is measured, not guessed at from a hunch.
- Pagination bounds the result set before performance becomes a real problem.
- An added index is itself a migration, reviewed the same as any schema change.
- Query performance is verified against real PostgreSQL, never against H2 or a mock.

KEY TAKEAWAY An index that exists in the migration but never gets used by the planner is exactly the kind of silent, undetected problem this review step exists to catch.

MODULE 50

Capstone Build:
Frontend and Persistence

Query Performance

Query performance measures how efficiently the database retrieves and returns data.

Good performance ensures fast response times, better scalability, and a superior user experience.

WHY QUERY PERFORMANCE MATTERS



Faster Response Times

Optimized queries return results quickly, improving application speed.



Better Scalability

Efficient queries handle more users and larger datasets.



Lower Resource Usage

Reduces CPU, memory, and I/O consumption on the database.



Better User Experience

Users get the data they need without delays or timeouts.



Cost Efficiency

Optimized queries reduce infrastructure costs and cloud spend.

FACTORS AFFECTING QUERY PERFORMANCE



Indexes

Proper indexes speed up data retrieval.



Query Structure

Efficient joins, filters, and aggregations matter.



Data Volume

Large tables require careful design and partitioning.



Statistics

Up-to-date statistics help the optimizer.



Hardware & Config

CPU, memory, I/O, and DB configuration impact speed.

QUERY EXECUTION FLOW



1. Parse

SQL is parsed and validated.



2. Plan

Optimizer creates the best execution plan.



3. Execute

Plan is executed to access data.



4. Return

Results are returned to the client.



The optimizer chooses the most efficient plan based on statistics, indexes, and available resources.

EXAMPLE: EXECUTION PLAN (PostgreSQL)

```
EXPLAIN ANALYZE SELECT * FROM employee
WHERE department_id = 10 ORDER BY salary DESC LIMIT 20;
```

Plan	Actual Time (ms)	Rows	Description
Limit	0.035...0.050	20	Returns top 20 rows
Sort	0.034...0.048	20	Sorts by salary DESC
Index Scan using idx_emp_dept_salary	0.012...0.028	100	Uses index lookup



Total execution time: 0.050 ms
Efficient plan using index and limit.

BEFORE AND AFTER OPTIMIZATION

BEFORE (Full Table Scan)

```
SELECT * FROM employee
WHERE department_id = 10;
```

Execution Plan	Actual Time
Seq Scan on employee	1200.345 ms
Filter: (department_id = 10)	1200.345 ms

Total Time: 1200.345 ms
Rows Returned: 10,532

AFTER (Index Scan)

```
SELECT * FROM employee
WHERE department_id = 10;
```

Execution Plan	Actual Time
Index Scan using idx_emp_dept	6.214 ms
Index Cond: (department_id = 10)	6.214 ms

Total Time: 6.214 ms
Rows Returned: 10,532

★ 99.48% improvement in execution time!

MEASURING TOOLS



EXPLAIN / EXPLAIN ANALYZE

Shows the execution plan and actual performance.



pg_stat_statements

Identifies slow and frequently executed queries.



Auto Explain

Logs execution plans of slow queries automatically.



Monitoring Tools

pgAdmin, Prometheus, Grafana, New Relic, DataDog.



Application Logs

Log query time at application level for end-to-end view.



BEST PRACTICES



Use indexes wisely.
Avoid over-indexing.



Write efficient SQL.
Avoid SELECT *.



Keep statistics
up to date.



Analyze execution
plans regularly.



Monitor and tune
continuously.



KEY TAKEAWAY

Measure, analyze, and optimize queries for consistent performance as data and usage grow.

ANGULAR → REST → SERVICE → REPOSITORY → POSTGRESQL FLOW

One user action, four checkpoints -- from a form submit to a durable, visible timeline update.

Stage	What Happens	Verified By
1. Validate (client)	Reactive Forms validators check required fields before the request is even sent.	Component test, empty summary
2. Call (typed service)	POST with X-Correlation-ID: lab-request-001, Content-Type json.	Network tab / test spy
3. Handle response	201 refreshes the timeline; 400/404/401 map to visible messages.	Component test per status
4. Persist (backend)	Module 49's service saves inside its own transaction.	Lab 49 evidence + SQL row

WHY IT MATTERS

1

1. Fill Form

Amina's interaction summary and channel, in the browser.

2

2. POST + Header

Typed service sends the request with the correlation header attached.

3

3. 201 + Refresh

Timeline refetches and shows the new interaction immediately.

4

4. SQL Proof

The row is visible in PostgreSQL and survives an API restart.

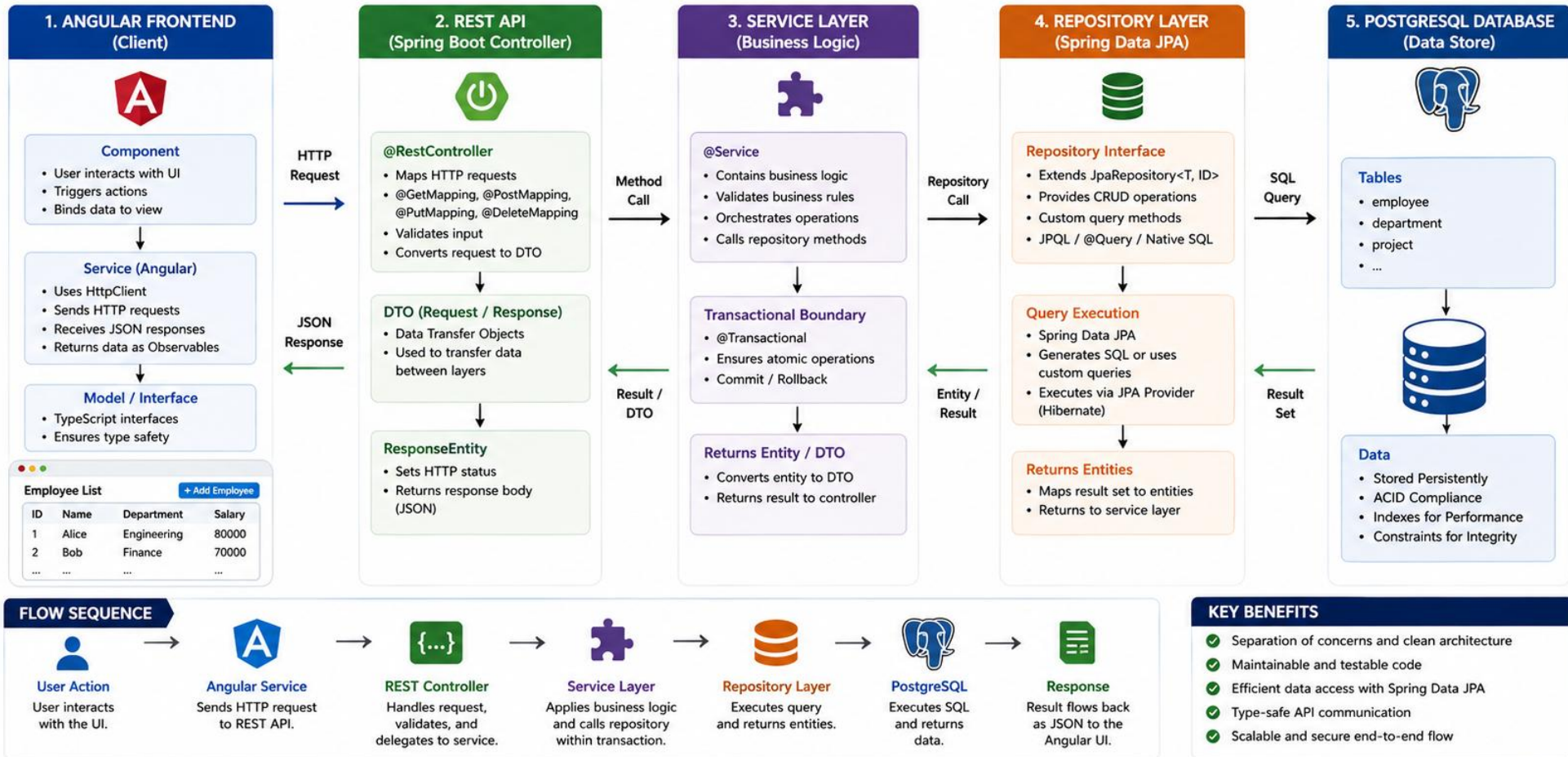
KEY TAKEAWAY A UI that shows success on a 400 response, or never refreshes the timeline after a real 201, breaks this workflow at exactly the checkpoint meant to catch it.

MODULE 50

Capstone Build:
Frontend and Persistence

Angular → REST → Service → Repository → PostgreSQL Flow

End-to-end data flow from the Angular UI to the PostgreSQL database through Spring Boot layers.



END-TO-END CREATE/READ/UPDATE/DELETE WORKFLOW

The same journey discipline applies to all four operations -- typed request in, refetch after, SQL proof at the end.



Create

InteractionForm posts a new interaction; 201 triggers a timeline refetch, not a hand-patched insert.



Read

Search and the timeline GET through the typed service; empty and not-found states are both explicit.



Update

An edit PATCHes with the entity's version field attached, for optimistic-locking safety.



Delete

A delete confirms first, then DELETes and refetches -- never a silent client-side row removal.

KEY TAKEAWAY Every one of the four operations follows the same rule: the UI trusts the server's response, never its own optimistic guess, once the request completes.

TRY IT YOURSELF — EXERCISE 5: TRACE ONE CRUD PATH

Reinforces: the Angular → REST → service → repository → PostgreSQL flow you just walked, traced by hand for a single CUS-1001 update.

STEPS (notes/lab50-e2e-crud-trace.md)

Step	What To Do
1 — Five Hops	Write the five hops for updating CUS-1001, naming the class or file at each.
2 — Transaction Edge	Say exactly where the transaction opens and where it commits.
3 — Proof of Success	Name the evidence that proves the write landed — not a green UI toast.
4 — Failure Path	Trace what the user sees when the update conflicts with a stale version.

Update path for CUS-1001

```
CustomerFormComponent -> CustomerService.update() -> PUT /api/customers/{id}
CustomerController -> CustomerService (@Transactional) -> CustomerRepository
commit -> 200 -> component refreshes; proof = SELECT shows the new row
```

TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-e2e-crud-trace.md (workspace: examples/module-50-exercises).

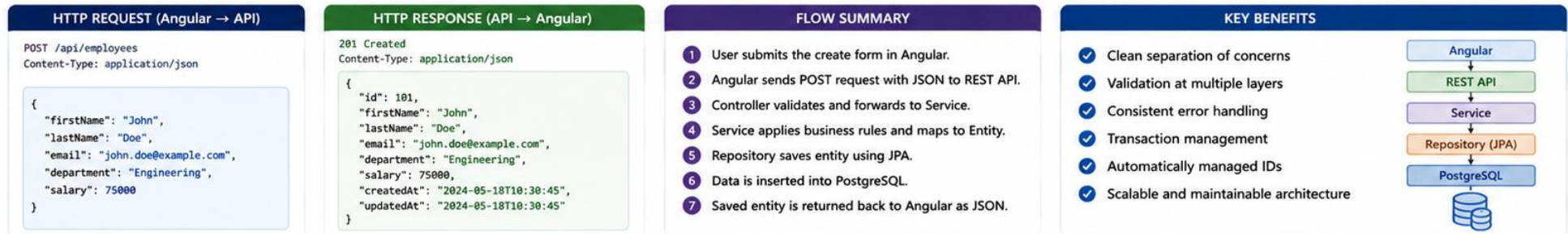
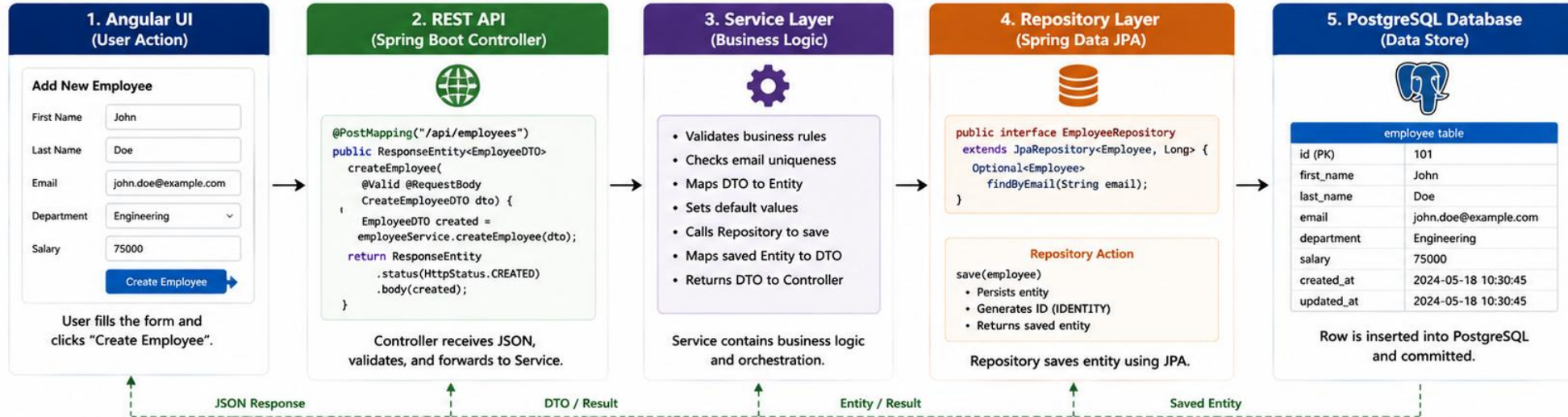
MODULE 50

Capstone Build:
Frontend and Persistence

End-to-End Create Workflow

This workflow shows how a new record (Employee) is created from the Angular UI to PostgreSQL database.

Angular → REST → Service → Repository → PostgreSQL.



Best Practice:

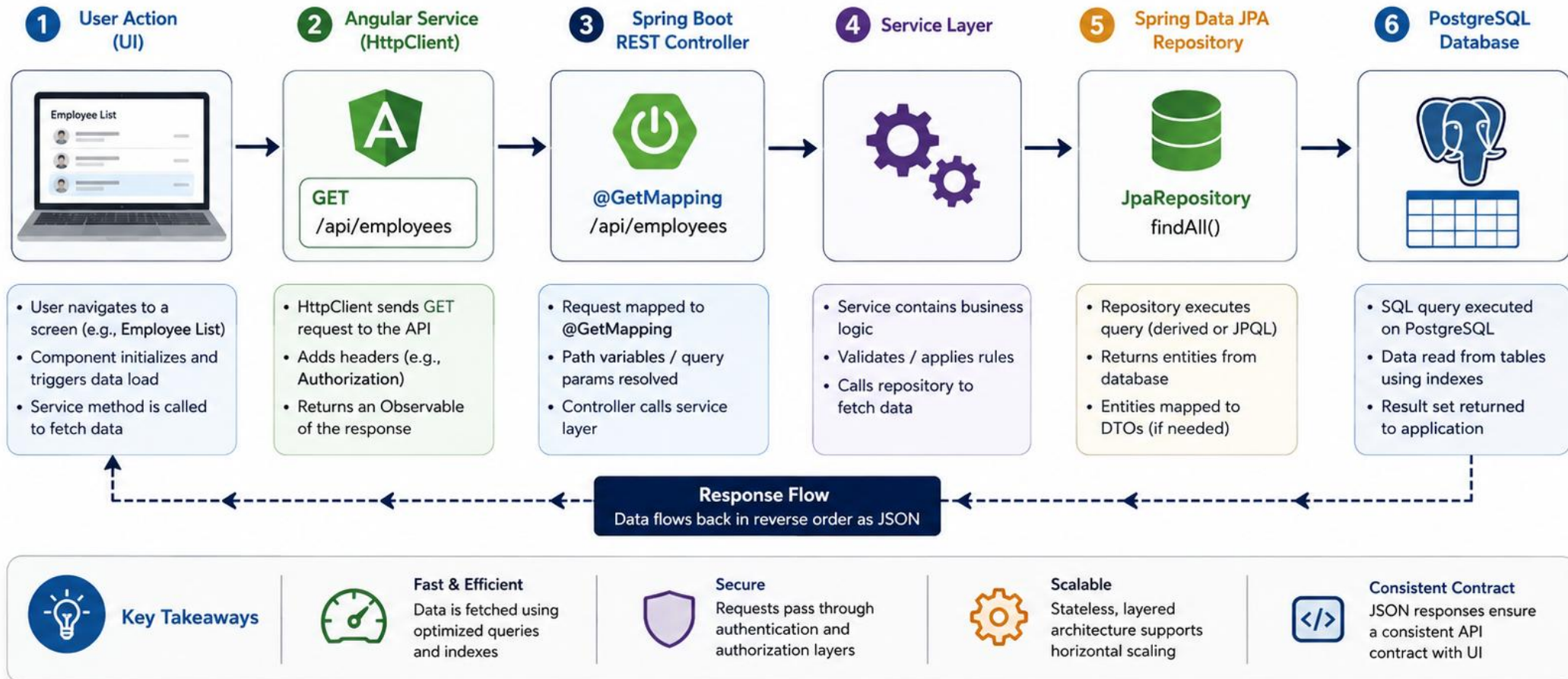
Always validate input, handle exceptions, and return meaningful responses.

Use DTOs to decouple API contracts from database entities.

End-to-End Read Workflow

User requests data → UI displays data

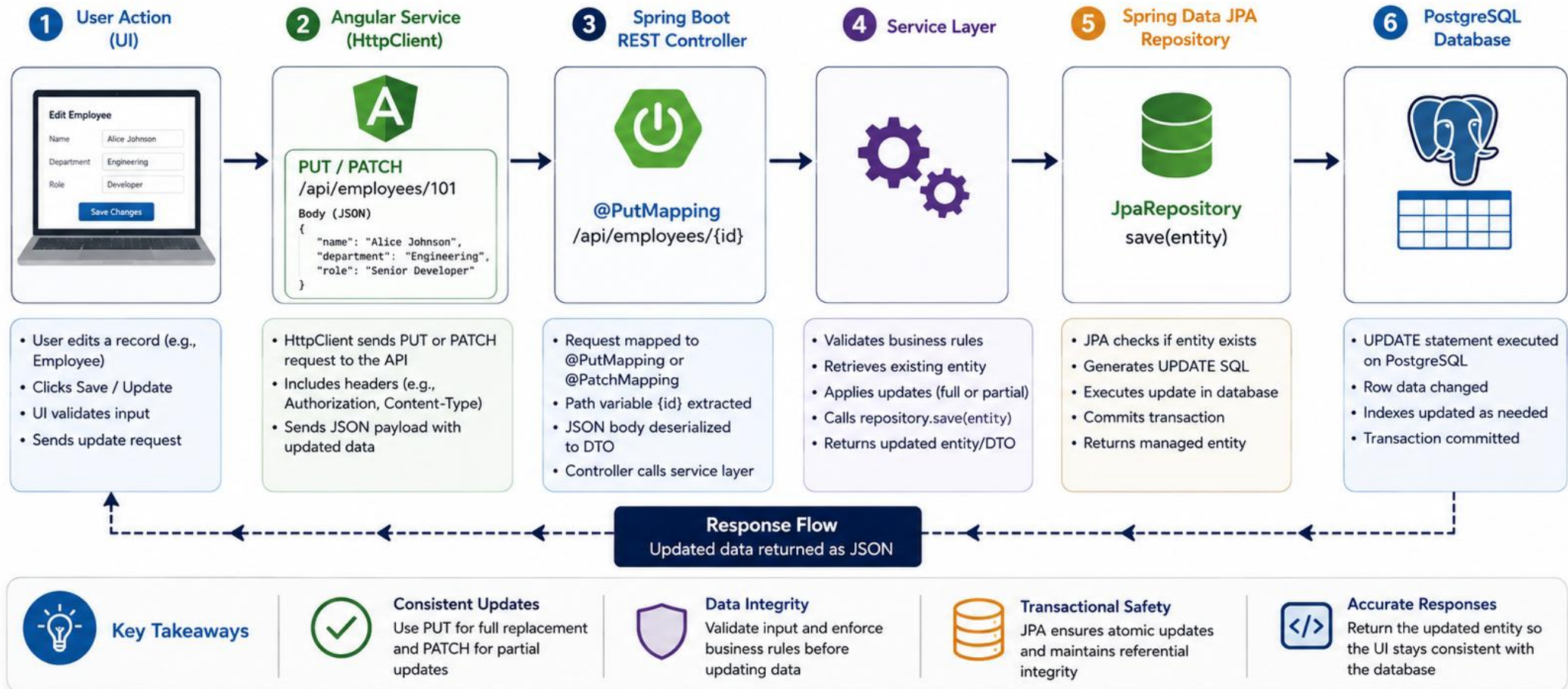
This flow shows how a Read operation (GET) moves from the Angular UI to PostgreSQL and back.



End-to-End Update Workflow

User updates data → Changes saved to database → Updated data returned to UI

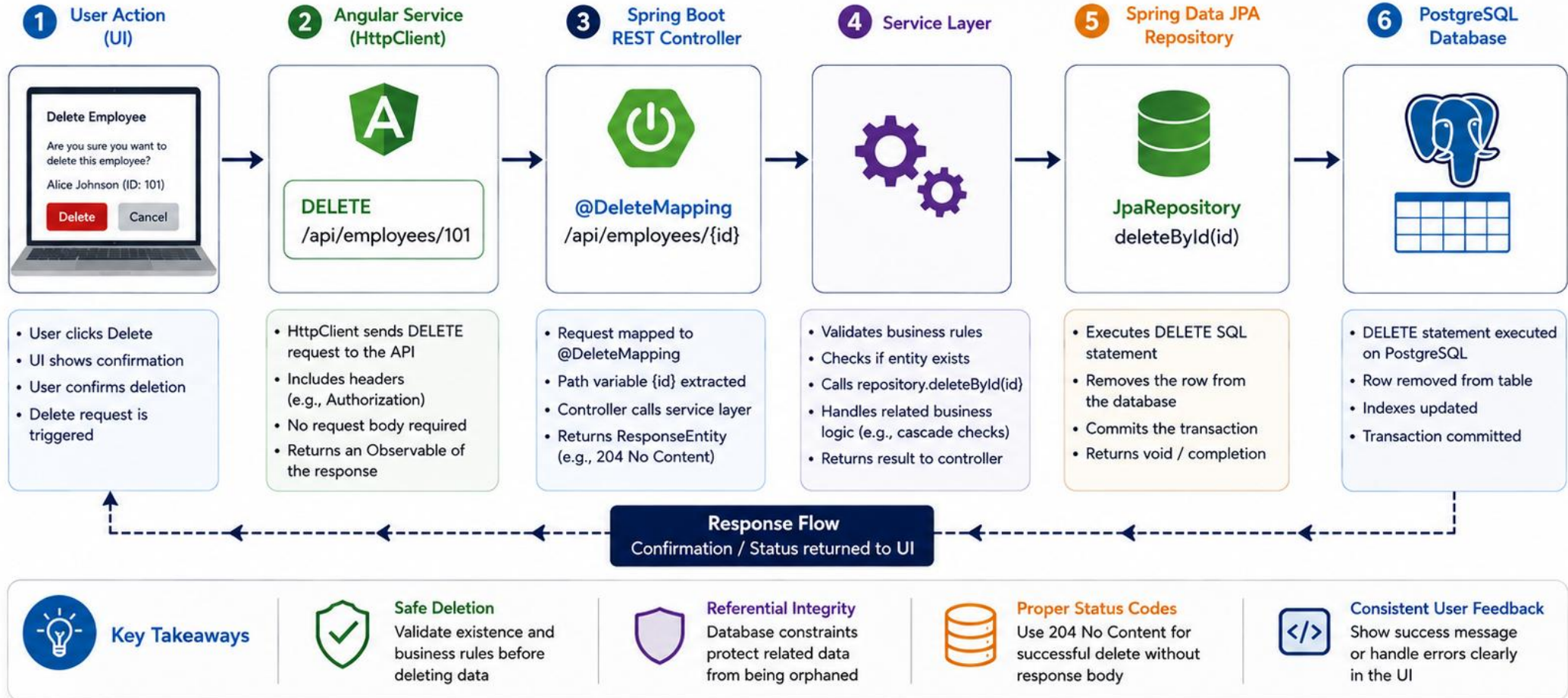
This flow shows how an Update operation (PUT/PATCH) moves from the Angular UI to PostgreSQL and back.



End-to-End Delete Workflow

User deletes data → Record removed from database → Confirmation returned to UI

This flow shows how a Delete operation (DELETE) moves from the Angular UI to PostgreSQL and back.



KAFKA-TRIGGERED UI UPDATES -- CONCEPTUAL INTEGRATION

Module 49's CustomerInteractionRecordedV1 event exists for downstream systems -- this module treats live UI push as a documented future step, not built today.

WHAT ALREADY EXISTS

Module 49 publishes CustomerInteractionRecordedV1 after every committed interaction.

The event is keyed by customer id and carries the correlation id end to end.

Today's Angular UI learns about a new interaction by refetching after its own POST succeeds.

No WebSocket or Server-Sent-Events push channel is built in this module.

Conceptual client sketch (not implemented this module)

```
// future: socket.on('interaction.recorded', evt => {  
//   if (evt.customerId === customerId()) timeline.refetch();  
// });
```

THE CONCEPTUAL PATH FORWARD

- A notification-service consumer could relay the event over WebSocket to subscribed clients.
- The Angular client would hold a live connection and patch the timeline signal on receipt.
- Idempotent handling still matters client-side -- a duplicate push must not double-render a row.
- This module documents the design; it does not implement live push - scope stays bounded.

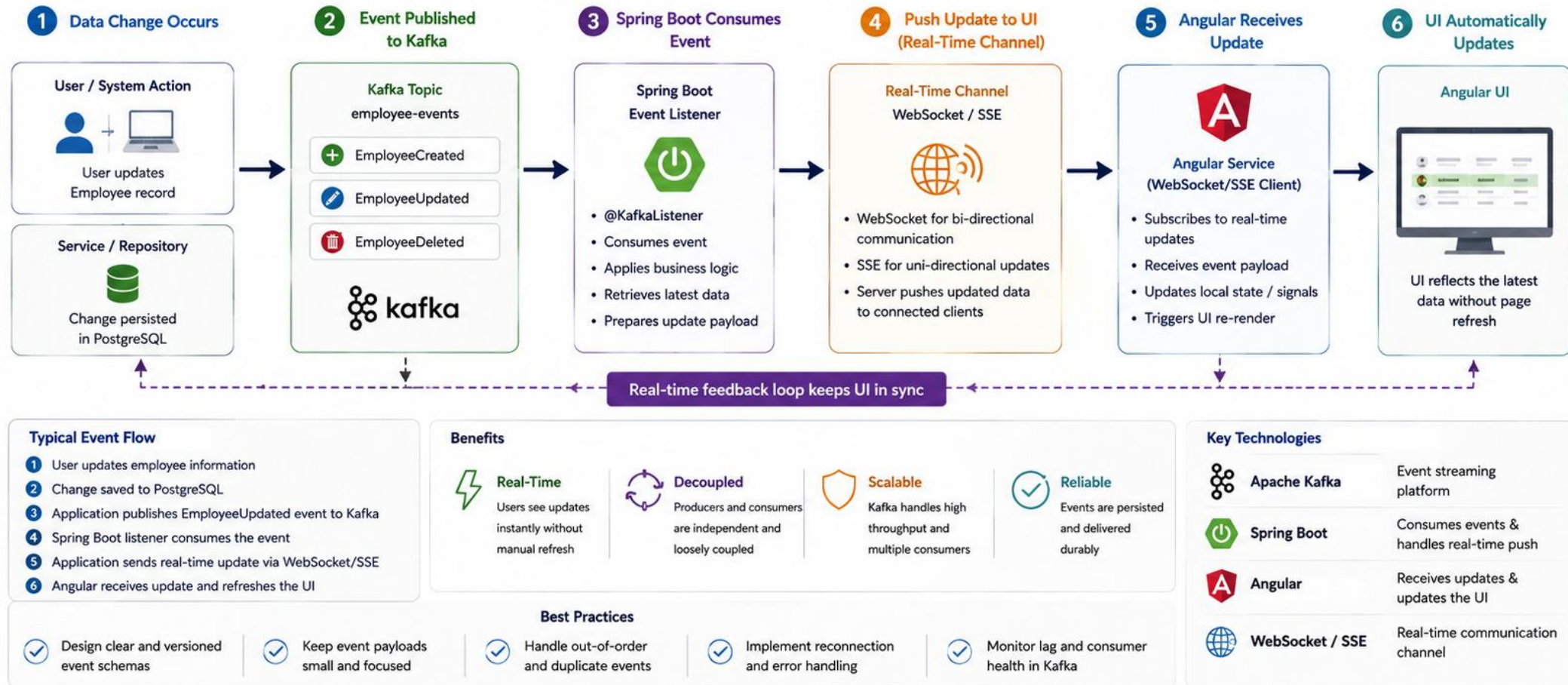
KEY TAKEAWAY Documenting a real, bounded integration point beats quietly building an unscoped feature -- live push stays a Module 52-defense talking point, not code shipped here.

Kafka-Triggered UI Updates – Conceptual Integration

Real-time data changes published to Kafka → Spring Boot consumes → UI updates automatically



Concept: When data changes occur (create/update/delete), events are published to Kafka. The backend consumes these events and pushes updates to the Angular UI via WebSocket/SSE for real-time synchronization.



FRONTEND UNIT TESTING

Component tests isolate rendering and validation logic -- fast, no network, no real backend needed.

WHAT COMPONENT TESTS COVER

InteractionForm validation -- required fields, max length.

InteractionTimeline rendering -- populated and empty states.

Error-state rendering for 400/401/404 responses.

Submit-button disabled state while a request is in flight.

TEST DISCIPLINE

- Use TestBed and accessible queries (By.css('[role=alert]'), label text), not brittle CSS class selectors.
- Mock the injected service, not HttpClient directly -- test the component's behavior.
- Assert visible text/roles via the DOM, not internal component state.
- Isolate test data -- do not depend on a shared, mutable fixture.

Component test (concept, Jasmine/Jest + TestBed)

```
const fixture = TestBed.createComponent(InteractionFormComponent);
fixture.componentInstance.customerId = signal('CUS-1001');
fixture.detectChanges();
fixture.debugElement.query(By.css('button[type=submit]')).nativeElement.click();
expect(fixture.debugElement.query(By.css('[role=alert]')).nativeElement.textContent)
  .toContain('required');
```

KEY TAKEAWAY A test coupled to a CSS class name breaks the moment a stylesheet changes -- accessible selectors are the same discipline this module's a11y rules already require.

Frontend Unit Testing

Test individual components, services, and functions in isolation for reliability and confidence



What is Unit Testing?



Testing a small piece of code in isolation (component, service, pipe, or utility function) to ensure it behaves as expected.

Why It Matters

- ✓ Catches bugs early
- ✓ Improves code quality
- ✓ Refactor with confidence
- ✓ Documents behavior
- ✓ Faster and cheaper than integration tests



Angular Testing Stack



Jasmine
Testing framework



Karma
Test runner

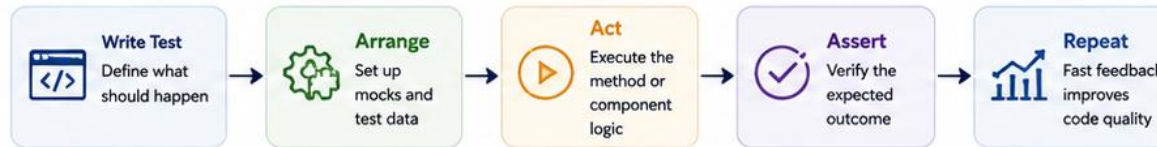


TestBed
Angular testing utility



Zone.js
Async test support

Unit Test Flow



Component Unit Test Example

Component (employee-card.component.ts)

```
@Component({
  selector: 'app-employee-card',
  template: `
    <div class="card">
      <h3>{{ employee.name }}</h3>
      <p>{{ employee.role }}</p>
    </div>`
})

export class EmployeeCardComponent {
  @Input() employee!: Employee;
}

export interface Employee {
  id: number;
  name: string;
  role: string;
}
```

Unit Test (employee-card.component.spec.ts)

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { EmployeeCardComponent, Employee } from './employee-card.component';

describe('EmployeeCardComponent', () => {
  let component: EmployeeCardComponent;
  let fixture: ComponentFixture<EmployeeCardComponent>;

  const mockEmployee: Employee = {
    id: 1,
    name: 'Alice Johnson',
    role: 'Software Engineer'
  };

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [EmployeeCardComponent] // standalone component
    }).compileComponents();
  });

  fixture = TestBed.createComponent(EmployeeCardComponent);
  component = fixture.componentInstance;
  component.employee = mockEmployee;
  fixture.detectChanges();
});

it('should display employee name and role', () => {
  const compiled = fixture.nativeElement as HTMLElement;
  expect(compiled.querySelector('h3')!.textContent).toContain('Alice Johnson');
  expect(compiled.querySelector('p')!.textContent).toContain('Software Engineer');
});
});
```



What to Test



Components
Inputs, outputs, rendering, events



Services
Methods, business logic, API calls (with mocks)



Pipes
Transformation logic



Utility Functions
Pure functions and helpers

Code Coverage



Target Coverage

- ✓ Branches
- ✓ Functions
- ✓ Lines
- ✓ Statements

Best Practices

- ✓ Test one thing at a time
- ✓ Use meaningful test names
- ✓ Keep tests independent and fast
- ✓ Mock external dependencies
- ✓ Follow AAA pattern (Arrange, Act, Assert)
- ✓ Focus on behavior, not implementation

Key Testing Utilities



TestBed
Creates Angular test environment



ComponentFixture
Access component instance & DOM



DebugElement
Query and interact with DOM



Jasmine Spies
Mock functions and verify calls

Helpful Commands



```
ng test
ng test --watch
ng test --code-coverage
ng test --browsers=ChromeHeadless
```

```
# Run unit tests once
# Run tests in watch mode
# Generate coverage report
# Run in headless browser
```

BACKEND PERSISTENCE TESTING

A repository or service test that never touches real PostgreSQL only proves the Java compiles -- not that the data survives.

WHAT TO TEST

@DataJpaTest loads Amina's seeded interactions and asserts the exact count and order.

A service test creates an interaction and asserts the returned DTO fields.

A negative test confirms CUS-9999 throws CustomerNotFoundException, mapped to 404.

A restart-durability check re-queries after simulating a fresh application context.

Repository test (concept)

```
@DataJpaTest
class InteractionRepositoryTest {
    @Test void findsAminasInteractionsNewestFirst() {
        var results = repo.findByCustomerIdOrderByCreatedAtDesc(AMINA_ID);
        assertThat(results).isNotEmpty();
    }
}
```

TEST DISCIPLINE

- Tests run against real PostgreSQL (Testcontainers or the shared training instance), not only H2.
- Each test seeds and cleans its own data -- no dependency on test execution order.
- Assertions check actual column values, not just 'no exception was thrown'.
- A green test suite with no restart-durability check has not proven this module's hardest requirement.

KEY TAKEAWAY A repository test that never actually runs against PostgreSQL only proves the Java method compiles -- it proves nothing about whether the underlying query is correct.

Backend Persistence Testing

Ensure data access logic, JPA mappings, transactions, and database integrity work as expected

Persistence Test Flow



What We Test

- ✓ Repository CRUD operations
- ✓ Custom query methods
- ✓ Entity relationships & mappings
- ✓ Cascade operations
- ✓ Transaction boundaries
- ✓ Constraints and validations
- ✓ Pagination & sorting
- ✓ Soft delete / audit fields

Why It Matters

- ✓ Catches data layer bugs early
- ✓ Prevents data integrity issues
- ✓ Safe to refactor persistence code
- ✓ Supports reliable deployments
- ✓ Improves application performance

Test Database Strategy

- Use a dedicated test database (separate from dev/prod)
- Auto-create schema (Hibernate) or use migrations
- Clean and repeatable state for every test run

Repository Integration Test Example

Entity

```
@Entity
@Table(name = "employees")
public class Employee {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String department;
    private String role;

    // getters and setters
}
```

Repository

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {

    List<Employee> findByDepartment(String department);
    List<Employee> findByRoleContainingIgnoreCase(String role);
    long countByDepartment(String department);
}
```

Test Class

```
@DataJpaTest
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Sql(scripts = "/data/seed-employees.sql")
class EmployeeRepositoryTest {

    @Autowired
    private EmployeeRepository repository;

    @Test
    void findByDepartment_returnsEmployees() {
        List<Employee> result = repository.findByDepartment("Engineering");
        assertFalse(result.isEmpty());
        assertEquals("Engineering", result.get(0).getDepartment());
    }

    @Test
    void countByDepartment_returnsCorrectCount() {
        long count = repository.countByDepartment("HR");
        assertEquals(2L, count);
    }
}
```

Key Annotations

- **@DataJpaTest** – configures JPA slice for repository tests
- **@Sql** – loads test data before tests
- **@Transactional (default)** – rolls back after each test
- **@AutoConfigureTestDatabase** – controls test DB behavior

Best Practices

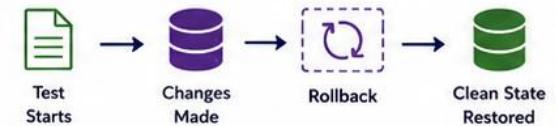
- ✓ Test one thing at a time
- ✓ Use meaningful test names
- ✓ Verify both positive and negative cases
- ✓ Keep tests independent and isolated
- ✓ Use test data builders or factories
- ✓ Avoid external dependencies

Test Architecture



Tests run against an isolated PostgreSQL test database using transactions for safety and repeatability.

Rollback for Safety



Tests are rolled back to keep data consistent and isolated.

Benefits

- | | |
|---|---|
| Data Integrity
Ensures rules, constraints, and relationships work | Faster Feedback
Quick tests catch issues before integration |
| Reliable Deployments
Safe changes with confidence | Performance Confidence
Validate queries and index usage |

Helpful Tools

- | | | | | |
|--|---|--|---|--|
| Spring Boot Test
Provides test context, auto-configuration | Spring Data JPA
Simplifies repository and persistence testing | Testcontainers
Spin up real PostgreSQL in Docker for tests | SQL Scripts
Seed and manage test data | DbUnit / AssertJ DB
Advanced database assertions |
|--|---|--|---|--|

SELENIUM UI TESTING

One real, critical-path journey through a real browser -- search Amina, save an interaction, see it on the timeline.

THE CRITICAL PATH

Search for Amina (CUS-1001) in a real, running browser against the Angular app.

Open her profile and timeline.

Submit a new interaction through the real, rendered form.

Assert the new interaction's text appears on the timeline.

E2E DISCIPLINE

- Stable selectors -- a real label, an ARIA role, or an explicit data-testid, never a brittle CSS path.
- Seed fixtures in setup -- do not depend on prior test-run state.
- Screenshots on failure only -- not on every single run.
- An instructor-approved substitute is acceptable if the E2E environment is unavailable.

Critical-path outline (concept)

```
driver.findElement(By.cssSelector("[data-testid=search-input]")).sendKeys("Amina");  
// ...select result, fill form, submit...  
assertThat(driver.findElement(By.cssSelector("[data-testid=timeline]")).getText())  
    .contains("Resolved login question");
```

KEY TAKEAWAY A Selenium test coupled to a fragile CSS class is a maintenance trap -- the same accessible-selector discipline from component tests applies here too.

Selenium UI Testing

Automate real user interactions in the browser to validate UI behavior and end-to-end scenarios

Test Execution Flow



What We Test

- ✓ UI components and elements
- ✓ Forms and user input
- ✓ Buttons, links, and events
- ✓ Navigation and routing
- ✓ Data display and tables
- ✓ Validation messages and errors
- ✓ Authentication and authorization flows
- ✓ End-to-end business workflows

Selenium Test Example (Java) – Login Test

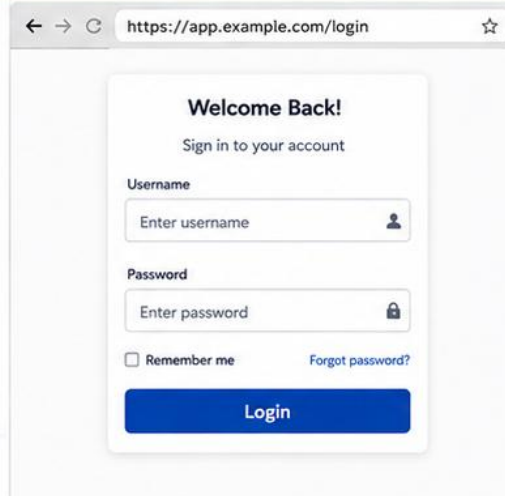
```
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;

public class LoginTest {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://app.example.com/login");

        // Enter username
        driver.findElement(By.id("username")).sendKeys("alice");
        // Enter password
        driver.findElement(By.id("password")).sendKeys("Password123");
        // Click login
        driver.findElement(By.cssSelector("button[type='submit']")).click();

        // Wait for dashboard and verify
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        WebElement welcome = wait.until(
            ExpectedConditions.visibilityOfElementLocated(By.id("welcome-msg")));
        assert welcome.getText().contains("Welcome, Alice");
        driver.quit();
    }
}
```

Application Under Test



Key Selenium Concepts

- Locators** Find elements using ID, Name, CSS, XPath, etc.
- Waits** Implicit waits, Explicit waits, Fluent waits
- Interactions** Click, SendKeys, Select, Hover, Drag & Drop, etc.
- Assertions** Verify text, attributes, visibility, values, and more
- Browser Management** Launch, maximize, navigate, and close browser

Best Practices

- ✓ Use stable locators (ID, data-test attributes)
- ✓ Avoid absolute XPath
- ✓ Use Explicit Waits (not Thread.sleep)
- ✓ Follow Page Object Model (POM)
- ✓ Keep tests independent and isolated
- ✓ Use meaningful test names
- ✓ Handle exceptions and take screenshots
- ✓ Run tests in headless mode in CI
- ✓ Keep tests fast and reliable

Handling Dynamic UI

- Explicit Waits
- Wait for Angular Stability
- Retry Mechanisms
- Check for Loading Indicators

Test Artifacts

- Screenshots on failure
- Logs and Console Output
- Test Reports (HTML)
- Videos (optional)

Running Selenium Tests

- Local Run
- Jenkins CI
- GitHub Actions
- Docker Container

Popular Tools & Libraries

- Selenium WebDriver
- TestNG
- WebDriver Manager
- Allure Reports



Tip: Keep UI tests focused on business flows, not implementation details. Reliable UI tests provide confidence in the user experience across browsers and environments.

END-TO-END TEST SCENARIOS

One critical path is required; a short, deliberate scenario list keeps coverage honest without chasing exhaustive UI permutations.

REQUIRED SCENARIOS

Happy path: search Amina, open profile, create an interaction, see it on the timeline.

Not-found: search CUS-9999, confirm the explicit not-found message renders.

Invalid input: submit an empty summary, confirm role=alert field error, no request sent.

Outage: stop the backend, confirm a real outage message, never a fake success.

Scenario checklist (excerpt)

```
[ ] Happy path (create)      [ ] Not-found (CUS-9999)
[ ] Invalid input (empty summary) [ ] Outage (backend stopped)
```

SCENARIO DISCIPLINE

- Each scenario maps to exactly one of the six states from the loading-and-empty-states slide.
- Scenarios are documented in docs/frontend-persistence-demo.md with expected evidence.
- A scenario that only 'usually passes' is not evidence -- it is re-run until it is reliable.
- New scenarios get added only when a real defect is found, not speculatively.

KEY TAKEAWAY Four deliberately chosen scenarios, each proven with real evidence, beat twenty scenarios nobody actually re-runs before every submission.

End-to-End Test Scenarios

Validate real user journeys across the full stack – UI, API, backend, database, and messaging



Purpose: Ensure the complete application works as expected from the user's perspective. These scenarios cover happy paths, validation, errors, security, and real-time updates.

Core End-to-End Test Scenarios

#	Scenario	Description	Key Steps (High Level)	Expected Result
1	User Login	Verify user can log in with valid credentials	Open app → Enter credentials → Click Login → Verify dashboard loads	✓ User logged in and dashboard displayed
2	Create Employee	Create a new employee record	Navigate to Employees → Click New → Enter details → Save → Verify in list	✓ Employee created and appears in the list
3	View Employee Details	View details of an existing employee	Search or select employee → Open details page → Verify all fields	✓ Employee details are shown correctly
4	Update Employee	Update employee information	Open employee → Click Edit → Update fields → Save → Verify updated data	✓ Changes saved and reflected correctly
5	Delete Employee	Delete an existing employee	Open employee → Click Delete → Confirm → Verify removed from list	✓ Employee deleted and no longer visible
5	Search, Filter & Sort	Search, filter and sort employees	Apply filters / sorting → Verify results → Clear filters → Verify reset	✓ Results match criteria and reset works
7	Authorization & Access Control	Ensure users can only access allowed resources	Login with different roles → Access restricted areas → Verify access allowed/denied	✓ Access control enforced correctly
8	Real-time Update (Kafka-driven)	Verify UI updates when backend event is published	Trigger event (create/update/delete) → Kafka publishes → Spring Boot consumes → UI updates in real time	✓ UI reflects real-time change without refresh
9	Error Handling	Validate system behavior on errors	Trigger API error / validation error → Verify error message → UI handles gracefully	✓ User sees meaningful error message
10	User Logout	Verify secure logout	Click Logout → Redirect to login → Try accessing protected URLs	✓ User logged out and access blocked

Business Benefits



High Confidence

Ensures the system works as a whole, not just in isolation



Catch Integration Issues

Finds defects in API contracts, data flow, and component interaction



Better User Experience

Validates real user journeys and business outcomes



Faster Releases

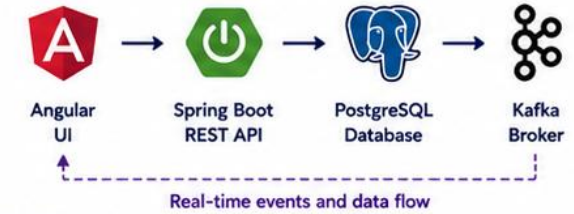
Automated E2E tests reduce manual testing effort and time

Best Practices

- ✓ Keep scenarios focused and independent
- ✓ Use stable test data and reset state between tests
- ✓ Avoid hard waits; use explicit waits and conditions

- ✓ Maintain tests as the application evolves
- ✓ Review and update tests with each feature
- ✓ Balance coverage and test execution time

System Under Test



Tools & Technologies



Selenium
WebDriver



JUnit 5
(Backend)



REST Assured
(API Tests)



Testcontainers
(PostgreSQL)



Kafka Test
Utils

Test Data & Environment

- ✓ Isolated test environment
- ✓ Dedicated PostgreSQL test database
- ✓ Seeded test data and cleanup after tests
- ✓ Kafka test topic for event scenarios
- ✓ Configurable via environment variables



Test Execution



Local Execution
(Developer)



CI Execution
(GitHub Actions)



Scheduled Runs
(Nightly/Regression)

INTEGRATION DEFECT RESOLUTION

When the Angular client and the Spring Boot API disagree, the fix starts with naming exactly where the contract broke.

TRIAGE STEPS

Reproduce with the network tab open -- confirm the actual request and response, not a guess.

Compare the response JSON field for field against docs/backend-demo.md.

Check the correlation id in the browser request and the backend log line -
- do they match?

Decide: is this a frontend interface drift, or did Module 49's contract actually change?

Triage checklist (excerpt)

```
[ ] Request URL and headers match the typed service
[ ] Response shape matches the interface
[ ] Correlation id present in both browser and backend log
```

COMMON ROOT CAUSES

- A TypeScript interface field name that no longer matches the DTO.
- An interceptor silently swallowing a 401 instead of surfacing it.
- A CORS or base-URL misconfiguration in environment.ts for the training cluster.
- A timezone or date-format mismatch between Instant and the Angular date pipe.

KEY TAKEAWAY A defect fixed by guessing and retrying is not resolved -- a defect fixed by naming the exact broken checkpoint is, and that checkpoint gets written down.



Integration Defect Resolution

Defects are inevitable in complex integrations. The key is a structured approach to identify, resolve, and prevent them from recurring.



1. IDENTIFY

Detect the defect through tests, logs, monitoring, or user feedback.



2. REPRODUCE

Reproduce the issue consistently and gather enough context and data.



3. ANALYZE

Analyze logs, traces, database state, and code flow to identify the root cause.



4. FIX

Implement the correct fix at the appropriate layer (UI, service, repository, or database).



5. VERIFY

Re-run tests and validate the fix in all impacted scenarios.



6. PREVENT

Add tests, improve validation, and update documentation to prevent similar defects.



KEY PRINCIPLES



COLLABORATE

Work together across the team



COMMUNICATE

Share findings clearly and early



DOCUMENT

Capture learnings and resolutions



IMPROVE

Strengthen tests and processes



Goal: Deliver reliable, high-quality integrations through a disciplined defect resolution process.



QUALITY TODAY, RELIABILITY TOMORROW.

PERFORMANCE REVIEW

A fast-looking demo and a genuinely responsive journey are not the same thing -- measure the parts that matter.



Time to Interactive Search

The search box must be usable within a glance, not blocked behind a slow initial bundle.



Bundle Size Awareness

Lazy-loaded routes keep the initial Angular bundle focused on the search screen.



No Redundant Refetches

The timeline refetches once per write, not on every unrelated signal change.



Backend Query Cost

The PostgreSQL timeline query is confirmed index-backed, not a silent sequential scan.

KEY TAKEAWAY A journey that feels fast in a demo but triggers three redundant HTTP calls per click has a real performance defect this review is meant to surface.



Performance Review

We evaluate the performance of the application end-to-end to ensure it meets quality, scalability, and user experience expectations under real-world conditions.



RESPONSE TIME

Measure API and UI response times. Aim for fast, consistent performance.



THROUGHPUT

Evaluate how many requests/users the system can handle.



RESOURCE UTILIZATION

Monitor CPU, memory, database connections, and JVM metrics.



DATABASE PERFORMANCE

Analyze slow queries, index usage, locks, and execution plans.



SCALABILITY

Validate system behavior under increasing load and concurrent users.

KEY METRICS DASHBOARD

AVG RESPONSE TIME

245 ms



THROUGHPUT

1,250 req/s



ERROR RATE

0.35 %

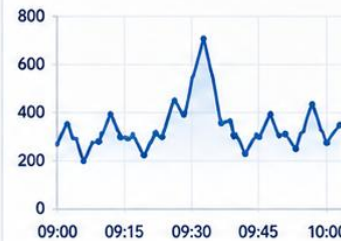


CPU USAGE

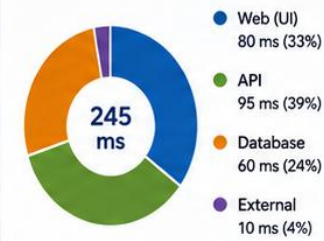
42 %



RESPONSE TIME (ms)



APM BREAKDOWN



TOP SLOW QUERIES

QUERY	AVG TIME
findAllEmployees	320 ms
getEmployeeById	210 ms
getDepartmentSummary	185 ms
findPaginatedEmployees	140 ms
saveEmployee	95 ms

PERFORMANCE CHECKLIST

- ✓ API response time < 300ms
- ✓ Error rate < 1%
- ✓ CPU usage < 70%
- ✓ No long-running database queries
- ✓ System stable under expected load



Goal: Ensure the application is fast, reliable, and scalable for real-world usage.



MEASURE. ANALYZE. OPTIMIZE. DELIVER EXCELLENCE.

ACCESSIBILITY REVIEW

Labels, keyboard order, and live-region alerts are rubric marks in this module, not a polish pass left for later.

WHAT GETS CHECKED

Every form field has a real <label for> bound to its input -- never placeholder-as-label.

Tab order moves logically: search -> results -> profile -> timeline -> form.

Errors announce through aria-live/role=alert, never color alone.

Focus moves somewhere sensible after a successful save, not left on a disabled button.

Accessible field (excerpt)

```
<label for="summary">Interaction summary</label>
<textarea id="summary" formControlName="summary" required maxlength="1000"
  [attr.aria-describedby]="errorId"></textarea>
@if (error()) { <p [id]="errorId" role="alert">{{ error() }}</p> }
```

REVIEW DISCIPLINE

- Run the journey keyboard-only, start to finish, with the mouse unplugged.
- Check color contrast on status badges (ACTIVE/PROSPECT) meets a readable minimum.
- CUS-9999 must render a readable not-found state reachable by a screen reader.
- Landmarks (main, nav) replace generic div soup so assistive tech can jump directly to search.

KEY TAKEAWAY A modal or form that traps keyboard focus forever, or an error only shown in color, fails the keyboard-only pass this module explicitly requires.



Accessibility Review

We ensure the application is usable by everyone, including people with disabilities, by following WCAG standards and building inclusive user experiences.

WCAG PRINCIPLES – POUR



PERCEIVABLE

Information must be presented in ways users can perceive.



OPERABLE

UI components must be usable and easy to operate.



UNDERSTANDABLE

Content and interactions must be clear and predictable.



ROBUST

Content must work across a wide range of assistive technologies and devices.

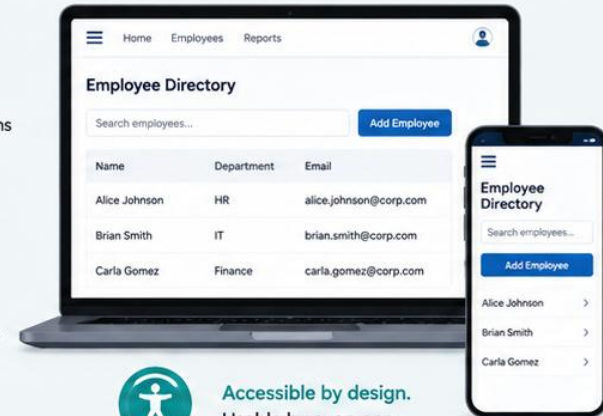


STANDARD COMPLIANCE



KEY CHECKPOINTS

- ✓ Text alternatives for images and icons
- ✓ Sufficient color contrast
- ✓ Keyboard navigability
- ✓ Focus indicators visible
- ✓ Form labels and instructions
- ✓ Semantic HTML and ARIA roles
- ✓ Error identification and suggestions
- ✓ Compatibility with screen readers



Accessible by design.
Usable by everyone.

COLOR & CONTRAST



Ensure minimum contrast ratio of 4.5:1 for normal text.

KEYBOARD ACCESS



All interactive elements must be reachable and operable via keyboard.

SCREEN READER SUPPORT



Use semantic markup, ARIA labels, and alt text to provide context for assistive technologies.

ACCESSIBILITY TOOLS

- ✓ axe DevTools
- ✓ WAVE Web Accessibility Evaluator
- ✓ Lighthouse Accessibility Audit
- ✓ NVDA / JAWS / VoiceOver



Remember: Accessibility is not a feature, it is a standard. Building accessible applications improves usability, SEO, and user satisfaction.



Goal: Deliver an inclusive application that everyone can access and use effectively.



INCLUSIVE DESIGN. BETTER EXPERIENCE. GREATER IMPACT.

SECURITY REVIEW

The Angular client is a consumer of Module 49's security boundary, never a replacement for it.



Client Validation Is UX Only

Reactive Forms checks help the user; the API still validates every field on every request.



No Token in Logs

A bearer token never appears in a console.log or a committed screenshot.



No Entity Leakage

No JPA entity or raw database column name ever appears in a rendered UI string.



401 Clears Session

An interceptor on 401 clears the stored token and redirects to login -- it never silently retries with a stale token.

KEY TAKEAWAY A security review that only checks the backend and assumes the frontend is fine misses exactly the class of leak this review exists to catch.



Security Review

We validate that the application protects data, prevents unauthorized access, and follows secure coding and configuration best practices.



AUTHENTICATION

Verify secure login, MFA (if applicable), and strong password policies.



AUTHORIZATION

Ensure role-based access control (RBAC) and least privilege access.



DATA PROTECTION

Validate encryption in transit (TLS) and at rest for sensitive data.



INPUT VALIDATION

Prevent injection attacks (SQL, XSS, CSRF) through validation and sanitization.



SECURE CONFIGURATION

Review server, application, and database configurations for secure defaults.



LOGGING & MONITORING

Ensure audit logs, alerting, and monitoring are enabled and reviewed.

SECURITY CHECKLIST

CHECKPOINT	STATUS	DETAILS
 Authentication	✓ PASS	JWT tokens, secure session management, password policies
 Authorization	✓ PASS	RBAC implemented, endpoint level access control
 Data Protection	✓ PASS	TLS 1.2+, encryption at rest, secrets managed securely
 Input Validation	✓ PASS	Input validation, output encoding, CSRF protection enabled
 Security Headers	✓ PASS	CSP, HSTS, X-Frame-Options, X-Content-Type-Options
 Database Security	✓ PASS	Parameterized queries, least privilege DB user
 Logging & Audit	✓ PASS	Audit logging enabled, sensitive data masked

THREAT COVERAGE



Injection (SQL/NoSQL)

✓ Mitigated



Cross-Site Scripting (XSS)

✓ Mitigated



Cross-Site Request Forgery (CSRF)

✓ Mitigated



Broken Access Control

✓ Mitigated



Sensitive Data Exposure

✓ Mitigated



Security Misconfiguration

✓ Mitigated

RISK SUMMARY



LOW RISK

No critical or high severity issues found.

 Critical	0
 High	0
 Medium	2
 Low	3
 Info	5

BEST PRACTICES FOLLOWED



Secure by Design



Least Privilege



Defense in Depth



Regular Vulnerability Scanning



Timely Patching & Updates



Security Awareness & Training



Goal: Deliver a secure, resilient application that protects users and data.



SECURE TODAY. PROTECT TOMORROW.

INTERNAL CODE REVIEW

Before this slice is called done, a second set of eyes checks it against the same rubric the lab is graded on.

REVIEW CHECKLIST

Does every component avoid the responsibility violations named on the component-strategy slide?

Does every write refetch instead of hand-patching state?

Are all six loading/empty/error states actually implemented, not just the happy path?

Does the PostgreSQL evidence include a restart, not just a live query?

Review comment style (example)

```
// Ref: entity-to-table-mapping slide -- version column missing @Version here.
```

REVIEW DISCIPLINE

- Review the diff, not just the final screen -- a passing demo can hide a wrong transaction boundary.
- Flag anything that silently diverges from Module 49's DTOs -- do not approve a workaround.
- A reviewer runs the keyboard-only pass personally, not just reads the code.
- Review comments reference the specific slide or ADR a change should match.

KEY TAKEAWAY A code review that only checks style and misses a missing @Version annotation or a hand-patched timeline has not actually reviewed this module's graded risks.



Internal Code Review

Internal code reviews ensure code quality, maintainability, security, and alignment with architecture and standards before changes are merged.

CODE REVIEW PROCESS



REVIEW FOCUS AREAS



APPROVAL CRITERIA

- ✓ All tests pass
- ✓ No critical or major issues
- ✓ Code follows standards
- ✓ Security best practices met
- ✓ Documentation updated
- ✓ PR description is clear

REVIEWER ROLES AND RESPONSIBILITIES

ROLE	RESPONSIBILITIES
Author	• Ensure code is complete and self-reviewed. • Provide clear PR description and context. • Address feedback promptly.
Reviewer	• Review code objectively and thoroughly. • Provide constructive, actionable feedback. • Validate changes meet standards and requirements.
Approver	• Ensure all feedback is resolved. • Confirm code meets quality, security, and architectural guidelines. • Approve and merge changes.

CODE REVIEW BEST PRACTICES



EXAMPLE REVIEW COMMENT

```
45 @@ public Employee createEmployee(EmployeeDto dto) {
46 - if (dto.getEmail() == null || dto.getEmail() == "") {
47 + if (dto.getEmail() == null || dto.getEmail().isBlank()) {
48     throw new IllegalArgumentException("Email is required");
49 }
```

Reviewer Comment
Good catch! Use `isBlank()` to handle empty and whitespace values. Also consider validating email format.

Severity: Minor
Type: Improvement



Goal: Deliver reliable, maintainable, and secure code through effective collaboration and review.



BETTER CODE TODAY, BETTER SOFTWARE TOMORROW.

PAIR PROGRAMMING REVIEW

Two roles, swapped regularly -- driver writes, navigator watches for exactly the mistakes this module names as graded.



Driver Writes

Types the component, service, or migration; narrates intent out loud.



Navigator Watches

Flags a missed label, an un-refetched write, or a cascade with no written reason -- in real time.



Swap on a Timer

Roles swap every 15-20 minutes so both partners write and both partners review.



Narrate the Why

Explaining a decision aloud catches more bugs than either partner working silently.

KEY TAKEAWAY A pair that never swaps roles, or navigates silently on a phone, gets none of the defect-catching benefit this practice actually provides.



Pair Programming Review

Pair programming improves code quality, knowledge sharing, and team alignment through real-time collaboration and constructive feedback.

PAIR REVIEW BENEFITS



PAIR PROGRAMMING REVIEW PROCESS



PAIR ROLES

DRIVER

Writes the code

- Focus on the implementation
- Think out loud while coding
- Ask questions and seek clarity
- Welcome feedback
- Make changes confidently



NAVIGATOR

Guides the direction

- Review the big picture
- Think about design, edge cases, and standards
- Spot risks and opportunities
- Provide feedback
- Ensure alignment with goals

Collaborate. Communicate. Create Better Code.

EFFECTIVE PAIR PROGRAMMING TIPS



PAIR REVIEW CHECKLIST

- | | |
|--|--|
| ☒ Requirements are fully understood | ☒ Tests written or updated |
| ☒ Code is readable and well-structured | ☒ No code duplication or dead code |
| ☒ Edge cases and error handling covered | ☒ Performance considerations addressed |
| ☒ Security and validation checks applied | ☒ Aligned with team standards and architecture |



Goal: Build better software together through collaboration, feedback, and shared ownership.



BETTER TOGETHER. BETTER CODE. BETTER PRODUCT.

AI-ASSISTED FRONTEND DEVELOPMENT

An AI assistant can draft an Angular component fast -- every draft still gets checked against this module's own rules before it ships.

WHERE AI HELPS

Scaffolding a standalone component's boilerplate -- imports, selector, template shell.

Suggesting a Reactive Forms validator pattern for a known field type.

Drafting a first-pass component test skeleton to fill in with real assertions.

Explaining an unfamiliar RxJS operator before it goes into a service.

WHAT STILL NEEDS A HUMAN

- Confirming the generated markup uses real labels, not an AI-plausible placeholder.
- Checking that a suggested HttpClient call matches Module 49's actual DTO field names.
- Rejecting any suggestion that hardcodes a token, a URL, or fake customer data as real.
- Verifying a generated test actually asserts a real outcome, not just 'no exception thrown'.

Prompting discipline (example)

```
"Generate a Reactive Forms group for CreateInteractionRequest  
(channel, summary) matching this exact TypeScript interface: ..."
```

KEY TAKEAWAY An AI-drafted component that looks complete but uses a placeholder instead of a label fails this module's accessibility rubric exactly like a human-written one would.



AI-Assisted Frontend Development

Leverage AI tools to accelerate development, improve code quality, and enhance user experience—while ensuring human oversight, security, and alignment with enterprise standards.

HOW AI HELPS IN FRONTEND DEVELOPMENT



AI IN YOUR ANGULAR WORKFLOW



EXAMPLE USE CASES

- ✓ Generate Angular components with inputs, outputs, and templates
- ✓ Create reactive forms with validation
- ✓ Convert API responses to TypeScript interfaces
- ✓ Refactor large components into smaller reusable pieces
- ✓ Optimize template performance (ngOnPush, trackBy)
- ✓ Improve accessibility and responsive design

AI PROMPT EXAMPLE



Prompt

"Create an Angular standalone component for an employee list with pagination, search, and sorting using Angular Material. Use TypeScript and include service integration."

AI Output (Example)

TS

```
@Component({
  selector: 'app-employee-list',
  standalone: true,
  imports: [CommonModule, MatTableModule, MatPaginatorModule,
    MatSortModule, MatInputModule, ReactiveFormsModule],
  templateUrl: './employee-list.component.html',
  styleUrls: ['./employee-list.component.scss']
})
export class EmployeeListComponent {
  dataSource = new MatTableDataSource<Employee>();
  displayedColumns: string[] = ['id', 'name', 'email', 'department'];
  searchControl = new FormControl('');
  constructor(private empService: EmployeeService) {}
  ngOnInit() {
    this.load();
    this.searchControl.valueChanges.subscribe(() => this.load());
  }
  load() { /* call API with pagination, sorting, filtering */ }
}
```

HUMAN OVERSIGHT IS ESSENTIAL

- ✓ Verify code correctness and business logic.
- ✓ Ensure security and validate inputs and outputs.
- ✓ Follow coding standards and best practices.
- ✓ Validate UI/UX consistency and accessibility.
- ✓ Test thoroughly before merging changes.

AI TOOLS YOU CAN USE



Goal: Use AI to augment productivity while maintaining quality, security, and accountability in frontend development.



BEST PRACTICE: AI assists, humans decide. Review, understand, test, and take ownership of every change.



AI-ASSISTED SQL/JPA REVIEW

An AI assistant can draft a migration or a repository method fast -- every draft still gets verified against real PostgreSQL before it ships.

WHERE AI HELPS

Drafting a first-pass Flyway migration for a new column or index.

Suggesting a Spring Data JPA method name for a described query.

Explaining an unfamiliar PostgreSQL feature (e.g., a partial index) before adopting it.

Reviewing a hand-written query for an obvious N+1 risk.

WHAT STILL NEEDS A HUMAN

- Running EXPLAIN ANALYZE on any AI-suggested query -- never trust an assumed index hit.
- Checking a suggested cascade or fetch type against this module's ownership rules.
- Verifying generated column types are PostgreSQL-native, not copied from another dialect.
- Confirming a suggested migration file number does not collide with one already applied.

Verification step (example)

```
EXPLAIN ANALYZE <ai-suggested query>; -- confirm index scan before accepting
```

KEY TAKEAWAY An AI-suggested migration that looks correct but uses the wrong vendor's type is exactly the kind of copied-example mistake this module's schema review already warns about.



AI-Assisted SQL/JPA Review

Use AI to improve query performance, ensure correctness, and follow best practices—while always applying human judgment and verification.

HOW AI HELPS



Query Analysis

Analyze SQL/JPA queries for issues and inefficiencies.



Performance Insights

Suggest optimizations, indexes, and query improvements.



Best Practices

Ensure queries and JPA usage follow enterprise standards.



Code Review

Review entity mappings, JPQL, and repository methods.



Explain & Learn

Explain complex queries and JPA behavior in simple terms.

AI REVIEW EXAMPLE – SQL QUERY OPTIMIZATION

Original Query

```
SELECT e.id, e.name, d.name AS dept, e.salary
FROM employee e
JOIN department d ON e.department_id = d.id
WHERE UPPER(e.name) LIKE '%SMITH%'
AND d.name = 'Engineering'
ORDER BY e.salary DESC;
```

AI Suggestions

- ✓ Avoid functions on columns (UPPER). Use ILIKE.
- ✓ Add index on department_id and name.
- ✓ Use covering index to improve sort.
- ✓ Consider pagination for large result sets.

Optimized Query (AI Suggestion)

```
SELECT e.id, e.name, d.name AS dept, e.salary
FROM employee e
JOIN department d ON e.department_id = d.id
WHERE e.name ILIKE '%smith%'
AND d.name = 'Engineering'
ORDER BY e.salary DESC
LIMIT 50 OFFSET 0;
```

Why Better?

- Uses ILIKE for case-insensitive search.
- Indexes can be used effectively.
- Supports pagination for performance.
- More scalable for large datasets.

AI REVIEW EXAMPLE – JPA QUERY & ENTITY MAPPING

Original JPQL

```
SELECT e FROM Employee e
JOIN FETCH e.department d
WHERE e.status = 'ACTIVE'
ORDER BY e.createdDate DESC
```

AI Findings

- Missing pagination.
- CreatedDate should be indexed.
- JOIN FETCH can cause N+1 with collections.
- Consider DTO projection for read-only use.

AI Improved JPQL

```
SELECT e FROM Employee e
JOIN FETCH e.department d
WHERE e.status = :status
ORDER BY e.createdDate DESC
Pageable pageable
```

AI Recommendations

- ✓ Use parameter binding (:status).
- ✓ Add index on (status, created_date).
- ✓ Use Pageable for large result sets.
- ✓ Use DTO projection if not updating entities.

AI-ASSISTED REVIEW WORKFLOW



1. Ask AI

Provide query/JPA code and context.



2. Analyze

AI analyzes logic, performance, and best practices.



3. Review

Review AI output, validate suggestions, and test changes.



4. Validate

Run tests and validate results and performance.



5. Approve

Approve and merge improvements into the codebase.

AI REVIEW CHECKLIST

- ✓ Query returns correct results
- ✓ Query is performant and scalable
- ✓ Indexes are used effectively
- ✓ No unnecessary SELECT *
- ✓ Joins are optimized
- ✓ Pagination used for large datasets
- ✓ Entity mappings are correct
- ✓ Fetch strategies are appropriate
- ✓ Transactions are well-defined
- ✓ Code follows JPA/SQL best practices
- ✓ Security (no SQL injection / unsafe input)
- ✓ Comments and naming are clear
- ✓ Changes are tested and validated



BEST PRACTICES



AI suggests, you decide. Always verify AI output.



Test every change with real data and edge cases.



Understand the query/JPA—don't rely blindly.



Document changes and rationale for future reference.



Share learnings with the team to build knowledge.

AI TOOLS YOU CAN USE



ChatGPT



GitHub Copilot



Amazon CodeWhisperer



Tabnine



JetBrains AI



Azure OpenAI



DBEAVER AI

Use AI as a powerful assistant—combined with domain knowledge and testing.



Goal: Deliver correct, performant, and maintainable SQL and JPA code through AI-assisted review and human expertise.



BETTER QUERIES. BETTER PERFORMANCE. BETTER APPLICATIONS.



VERIFYING AI-GENERATED CHANGES

Whether AI drafted a component, a migration, or a test, the same verification bar applies as any other change in this module.

VERIFICATION STEPS

Read every line before accepting it -- an AI suggestion is a draft, not a merge-ready patch.

Run the actual test suite; a suggestion that 'looks right' is not evidence.

Check the diff against the real DTO, entity, or migration it claims to match.

Confirm no synthetic-looking placeholder secret, token, or fake customer PII slipped in.

Verification log entry (example)

```
AI drafted InteractionForm validators; verified against CreateInteractionRequest; 2 fields corrected.
```

RECORD-KEEPING

- Note in docs/frontend-persistence-demo.md where AI assistance was used and what was changed by hand.
- A rejected AI suggestion is worth noting too -- it documents a checked, ruled-out approach.
- AI assistance never substitutes for the restart-durability or accessibility evidence this module requires.
- The same rubric applies regardless of who -- or what -- wrote the first draft.

KEY TAKEAWAY An AI-assisted change that was never actually run against real PostgreSQL or a real browser is unverified, no matter how confident the suggestion looked.

16

Verifying AI-Generated Changes



Trust, but verify. You own the quality, security, and correctness.



1. Understand the Change

Read the AI explanation and the code carefully. Ensure you understand what was changed and why.



2. Validate Correctness

Confirm the change meets requirements and handles edge cases. Add or update unit tests.



3. Check Security

Review for vulnerabilities, unsafe practices, and data exposure. Follow secure coding standards.



4. Review Performance Impact

Look for inefficient queries, loops, or memory usage. Consider scalability and resource consumption.



5. Ensure Consistency

Verify alignment with project conventions, architecture, and existing patterns.



6. Test and Validate

Run automated tests and manual checks. Validate in local and integration environments.



You are responsible for every line that goes into production.

</> Example: Review AI-Generated Code Diff

AI Suggestion (Before)

```
1 @GetMapping("/users/{id}")
2 public User getUser(@PathVariable
3     Long id) {
4     return userRepository.findById(id)
5         .get();
6 }
7
8 // No error handling
```



Your Review (After)

```
1 @GetMapping("/users/{id}")
2 public ResponseEntity<User> getUser(
3     @PathVariable Long id) {
4     return userRepository.findById(id)
5         .map(user -> ResponseEntity.ok(user))
6         .orElse(ResponseEntity
7             .notFound().build());
8 }
9 // Proper error handling added
```

What to Verify

- ✓ Handles missing user gracefully
- ✓ Uses appropriate HTTP status codes
- ✓ Follows REST best practices
- ✓ No potential exceptions (.get() removed)
- ✓ Consistent with project standards
- ✓ Covered by unit/integration tests



Key Reminder

AI accelerates development, but human judgment ensures quality. Always review, test, and take ownership of the final change.



CAPSTONE BUILD CHECKLIST

Before calling this slice done, run this checklist -- it is the definition of a demoable, defensible frontend-and-persistence build.



UI Evidence

Search, profile, timeline, and form work end to end for Amina and Ravi; CUS-9999 shows not-found.



Persistence Proof

Migration applied; the row exists; it survives an API restart.



Accessibility Basics

Labels, keyboard order, and role=alert/aria-live all verified by a keyboard-only pass.



Automated Check

Component tests plus one Selenium critical-path journey, both green.

KEY TAKEAWAY No pass on this build without UI proof, PostgreSQL row proof, accessibility basics, and at least one automated UI or component check on the critical path.

TRY IT YOURSELF — EXERCISE 6: PLAN TESTS AND DEMO EVIDENCE

Capstone pre-lab gate: name the test scenarios and demo evidence Lab 50 grades, then confirm the five earlier plans are genuinely complete.

STEPS (notes/lab50-test-demo-readiness.md)

Step	What To Do
1 — Test Matrix	List one Angular unit test, one repository test, and one Selenium scenario.
2 — Failing Case	Include at least one failure scenario, such as CUS-9999 not found.
3 — Demo Evidence	Name the screenshots and the notes file the demo pack will contain.
4 — Pass Mark	Mark Pass only if all five earlier notes files are complete, not stubbed.

Minimum test matrix

```
unit      CustomerService maps 404 to the empty-state signal
repository interaction history for CUS-1001 returns rows in order
selenium   create CUS-1001, read it back, edit it, see it persist
```

TRY IT NOW — Complete Exercise 6 before Lab 50: exercises/exercise-06-test-and-demo-readiness.md (workspace: examples/module-50-exercises).



49 Capstone Build Checklist

Use this checklist to ensure a high-quality, enterprise-ready capstone solution.

 1 Requirements & Design ✓ Business requirements are clearly understood ✓ Architecture and data flow are documented ✓ ERD and API contracts are defined ✓ Technology stack and tool versions are finalized	 2 Backend (Spring Boot) ✓ REST endpoints follow standards and naming ✓ Input validation and error handling implemented ✓ Service layer and business logic complete ✓ Unit and integration tests written and passing ✓ API documentation (OpenAPI) is up to date	 3 Frontend (Angular) ✓ Components are modular and reusable ✓ Services communicate with REST APIs ✓ Forms have validation and error handling ✓ Loading, empty, and error states implemented ✓ Responsive UI and accessibility considered
 4 Database (PostgreSQL) ✓ Schema is normalized and well designed ✓ Constraints, indexes, and relationships defined ✓ Migrations are versioned and repeatable ✓ Seed or reference data is available ✓ Queries are optimized and tested	 5 Integration & Messaging (Kafka) ✓ Kafka topics, producers, consumers configured ✓ Events are published and consumed correctly ✓ Retry and error handling implemented ✓ Integration tested end-to-end	 6 Testing ✓ Unit tests have good coverage ✓ Integration tests verify key flows ✓ UI tests (Selenium) pass reliably ✓ Edge cases and negative scenarios tested ✓ All tests pass in CI pipeline
 7 Security ✓ Authentication and authorization implemented ✓ JWT handling and token validation secured ✓ Sensitive data encrypted/secured ✓ Input validation and SQL injection prevention ✓ Dependencies are up to date and scanned	 8 CI/CD Pipeline ✓ Builds succeed for frontend and backend ✓ Tests run automatically on PRs ✓ Code quality and security scans pass ✓ Artifacts are published and versioned ✓ Deployment to OpenShift is successful	 9 Documentation & Operations ✓ README with setup and run instructions ✓ API documentation is complete ✓ Operational runbooks are available ✓ Monitoring and logging are configured ✓ Rollback strategy is documented and tested
 Definition of Done:  All checklist items completed  Code reviewed and approved  Security and quality gates passed  Deployed to staging successfully  Ready for production approval		

LAB OVERVIEW — CAPSTONE BUILD — ANGULAR FRONTEND AND POSTGRESQL PERSISTENCE

Angular feature routes · HttpClient + JWT interceptor · Flyway/schema · E2E CRUD · Selenium notes /

BUSINESS SCENARIO

- Agents need a UI to manage interactions for Amina (`CUS-1001`) with data durable in PostgreSQL. Leadership freezes: No Lab 50 pass without Angular CRUD evidence, migration-backed schema, JWT-authenticated API calls, and a demo runbook.

LEARNING OBJECTIVES

- Structure Angular features with routing and reusable components
Call REST APIs via `HttpClient` services and a JWT interceptor
Align DTOs with Spring Boot contracts
Implement JPA entities/repositories and Flyway migrations on PostgreSQL
Demonstrate E2E CRUD and document Selenium scenarios

WHAT YOU BUILD

- Angular feature module/structure, routes, components; Services using HttpClient + JWT interceptor; PostgreSQL schema / JPA entities / repositories for the slice; Flyway (or approved) migrations + seed notes

KEY TAKEAWAY This Module 50 lab implements the ****Angular frontend**** and ****PostgreSQL persistence**** for the Northstar CRM capstone slice planned in Lab 48 and backed by Lab 49 APIs: feature structure, routing, services, JWT interceptor, schema/entities/repos, migrations, CRUD...

Lab Overview – Capstone Frontend and Persistence



Build the Angular frontend and connect it to Spring Boot and PostgreSQL with JPA for full CRUD capability.



Lab Goal

Implement a responsive Angular application that consumes Spring Boot REST APIs and performs CRUD operations using Spring Data JPA with PostgreSQL persistence.



What You Will Build

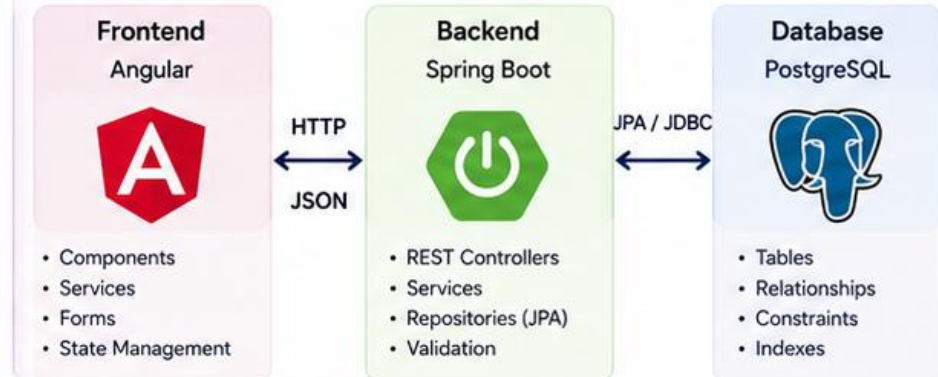
- ✓ Angular UI for managing capstone domain data
- ✓ REST API integration with HttpClient
- ✓ Service layer and DTO usage
- ✓ Spring Data JPA repositories
- ✓ PostgreSQL database with relationships
- ✓ End-to-end create, read, update, delete flows



Technology Stack

-  Angular 17+
-  Spring Boot 3.x
-  Spring Data JPA
-  PostgreSQL 15+
-  Java 17+, Maven

Architecture Overview



Key Objectives

- ✓ Develop modular Angular components and forms
- ✓ Integrate Angular with Spring Boot REST APIs
- ✓ Implement CRUD operations end-to-end
- ✓ Map DTOs and handle validation errors
- ✓ Persist and retrieve data using JPA repositories
- ✓ Ensure UI reflects backend data accurately



Key Flows to Implement



Lab Activities

- 1 Setup Angular project and create modules/components
- 2 Build forms and UI for entities
- 3 Create Angular services and connect to REST APIs
- 4 Implement CRUD operations and handle responses
- 5 Implement backend repositories and entities
- 6 Test database integration and relationships
- 7 Handle validation and error scenarios
- 8 End-to-end testing and UI verification



Deliverables

- ✓ Angular application with working CRUD UI
- ✓ Integrated REST APIs and services
- ✓ JPA entities, repositories, and database schema
- ✓ PostgreSQL database with sample data
- ✓ Documentation of setup and API usage



This lab focuses on building a fully functional frontend and persistence layer with real-world integration and data flows.

LAB TASKS AND DELIVERABLES

Capstone Build — Angular Frontend and PostgreSQL Persistence — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Open starter/README.md.
2	Copy into java-bootcamp/examples/lab50-capstone (or your customer-management-platform monorepo—see starter).
3	Fill Angular service/interceptor TODOs and confirm Flyway migration TODO for the slice.
4	Smoke: list/create against local API + PostgreSQL; evidence under notes/screenshots/lab-50/.
5	Mark timed-path Pass criteria. Continue remaining GUIDE steps as homework.
6	Angular feature module/structure, routes, components
7	Services using HttpClient + JWT interceptor
8	PostgreSQL schema / JPA entities / repositories for the slice
9	Flyway (or approved) migrations + seed notes
10	E2E CRUD evidence for CUS-1001 (create/read/update/delete or soft-delete as designed)
11	Selenium (or Playwright-if-approved) scenario notes
12	docs/frontend-persistence-demo.md

EXPECTED DELIVERABLES

- Angular feature module/structure, routes, components Services using HttpClient + JWT interceptor PostgreSQL schema / JPA entities / repositories for the slice Flyway (or approved) migrations + seed notes E2E CRUD evidence for CUS-1001 (create/read/update/delete or soft-delete as designed) Selenium (or Playwright-if-approved) scenario notes docs/frontend-persistence-demo.md No secrets, real PII, or Oracle/React as...

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY Lab 50 is graded capstone work. Pre-lab exercises 1→6 must be Pass before the lab path; keep evidence under notes/screenshots and never commit secrets.

51 Lab Tasks and Deliverables

Complete the tasks below to build the Angular frontend and persistence layer of the capstone application.



Lab Tasks

1		Project Setup	Clone the repository, set up Angular and Spring Boot projects, and run both applications locally.
2		Database & Entities	Review the PostgreSQL schema, create entities, and configure Spring Data JPA repositories.
3		CRUD REST APIs	Implement REST endpoints for create, read, update, and delete operations with validation.
4		Angular Modules & Components	Create feature modules and components for managing the domain data.
5		Service Layer Integration	Build Angular services using HttpClient to call the REST APIs and handle responses.
6		Forms & Validation	Implement reactive forms with validation and display backend validation errors.
7		Data Display & UX States	Implement lists with pagination, sorting, and filtering. Add loading, empty, and error states.
8		End-to-End Flow Testing	Test full CRUD flows end-to-end and verify data persistence in PostgreSQL.



Success Tip: Commit your work frequently, test as you build, and ensure both frontend and backend are running before moving to the next task.



Deliverables

✓	Angular Application	Working Angular application with all modules, components, and routes.
✓	REST API Implementation	Complete REST API for CRUD operations with proper validation and error handling.
✓	JPA Entities & Repositories	Entity classes, relationships, and Spring Data JPA repositories.
✓	Database Schema	PostgreSQL schema (migrations or SQL scripts) and sample seed data.
✓	UI Features	Forms, lists with pagination/sorting/filtering, and UI state handling.
✓	End-to-End Workflow	All CRUD flows working end-to-end with data persisted in PostgreSQL.
✓	Test Coverage	Backend repository tests and basic frontend unit tests.
✓	Documentation	Setup instructions, API usage guide, and notes on key decisions.



Definition of Done

- All tasks completed and tested
- Code committed to GitHub
- Documentation updated
- Application runs without errors end-to-end

MODULE 50 SUMMARY

In this module, we completed the Northstar CRM journey end to end: an accessible Angular UI backed by proven PostgreSQL persistence.

KEY CONCEPTS COVERED



Connected Journey

Search, profile, timeline, and form as one flow, not isolated components.



Typed API Services

One injectable service per resource, cancellation-safe, correlation header always sent.



Accessibility Graded

Real labels, keyboard order, and role=alert/aria-live -- not color-only feedback.



PostgreSQL Proof

Versioned migrations, reviewed JPA mappings, restart-durability evidence.



Full State Matrix

Loading, empty, success, invalid, unauthorized, and outage -- all six.



Tests as Evidence

Component tests plus one critical-path Selenium journey.

MODULE FLOW RECAP

1

1. Map Journey

2

2. Build UI + Services

3

3. Wire Persistence

4

4. Handle All States

5

5. Prove + Automate

KEY TAKEAWAY A journey is graded end to end: connected flow, typed services, accessibility, PostgreSQL proof, full state handling, and automated tests -- all six, not just the happy path.

52 Module Summary

You built the frontend and persistence layer of the capstone application.



 **What We Built**

**Angular Frontend**
Modular, responsive UI with components, services, routing, forms, and state management.

**Backend Integration**
Connected Angular with Spring Boot REST APIs using HttpClient and typed models.

**Persistence Layer**
Designed PostgreSQL schema and implemented entities with Spring Data JPA repositories.

**CRUD Functionality**
Implemented end-to-end create, read, update, delete flows with validation and error handling.

**Data Handling**
Added pagination, sorting, filtering, loading, empty, and error states for better UX.

**Data Persistence**
Verified that all data operations persist correctly in PostgreSQL with JPA.

 **Key Takeaways**

**Full-Stack Integration**
Integrated Angular frontend with Spring Boot APIs and PostgreSQL persistence.

**Layered Architecture**
Worked across UI, service, controller, service layer, repository, and database layers.

**DTO Alignment**
Ensured consistent data contracts between frontend and backend.

**Enterprise Best Practices**
Applied validation, error handling, pagination, and clean code practices.

**End-to-End Confidence**
Built, tested, and verified complete CRUD workflows with real data.

 **Outcomes**

**Functional Application**
A working Angular application integrated with Spring Boot and PostgreSQL.

**Reliable Persistence**
Data is stored, retrieved, and updated reliably in the database.

**Scalable Foundation**
Well-structured codebase ready for advanced features and scaling.

**Production Readiness**
Follows best practices for quality, security, and maintainability.

 **You Now Have**

 A responsive UI

 Integrated REST APIs

 Validated business logic

 Persistent data in PostgreSQL

 A strong foundation for the next modules

 **What's Next?**

In the next module, we will **connect Kafka messaging** and implement event-driven updates to enhance real-time capabilities.

 Frontend

→

 Backend

→

 Database

→

 Kafka

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of the typed service layer, accessibility, and state handling.

1. Where should every network call in the Angular journey originate?

- A. Directly inside each component's template
- B. A single, centralized injectable service**
- C. A global variable set once at startup
- D. Inline inside an RxJS operator with no service

3. What actually proves a UI write is persistent, per this module's rule?

- A. A success toast appearing
- B. A SQL row that survives an API restart**
- C. The browser network tab showing 201
- D. The developer's memory that it worked once

2. What must replace placeholder text as a form field's label?

- A. A tooltip
- B. A real <label for> element**
- C. A color change on focus
- D. Nothing -- placeholders are sufficient

4. What should the UI do when the API returns a 400 Problem Details response?

- A. Show a success message anyway
- B. Show field-level errors via role="alert"**
- C. Silently ignore the response
- D. Retry the same request automatically forever

KEY TAKEAWAY Centralize network calls in one typed service; real labels beat placeholders; restart-surviving SQL rows are the only persistence proof; 400s map to accessible field errors.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on persistence discipline, fixtures, testing, and the merge gate.

5. Which fixture ID exercises the not-found empty state in Lab 50?

- A. CUS-1001
- B. CUS-1002
- C. CUS-9999**
- D. lab-request-001

7. Per this module's testing scope, what does the required Selenium journey cover?

- A. Every possible UI state exhaustively
- B. One real critical path: search, save, verify the timeline**
- C. Only the loading spinner's CSS animation
- D. Backend unit tests exclusively

6. What must happen before a JPA entity can safely map to a database column?

- A. Nothing -- Hibernate infers it automatically and safely
- B. A versioned migration must already define that column**
- C. The frontend must call the API once
- D. The column name must match a Java keyword

8. What is required before this build can be marked a pass, per this module's rule?

- A. A single successful screenshot
- B. UI proof, DB proof, accessibility, and one automated check**
- C. Only a green ng build
- D. Instructor sign-off with no evidence

KEY TAKEAWAY CUS-9999 is the not-found fixture; migrations precede entity mapping; Selenium covers one critical path; a pass requires all four forms of evidence together.

*"A component tree that renders is not proof
-- only a PostgreSQL row surviving a restart
is."*

MODULE 50 COMPLETE

Capstone Frontend and Persistence

You can now build an accessible Angular journey -- search, profile, timeline, and form -- typed against a real API, backed by PostgreSQL persistence you can prove survives a restart, with component and Selenium tests covering the critical path.